3 Il ciclo di vita del software rivisitato

OBIETTIVI FORMATIVI

- Essere a conoscenza di una serie di modelli generici per strutturare il processo di sviluppo del software
- Apprezzare i pro ei contro di questi modelli, in particolare quelli dei metodi agili e orientati alla pianificazione
- Comprendere le somiglianze tra la manutenzione del software e l'evoluzione del software
- Riconoscere che è vantaggioso applicare l'ingegneria della linea di prodotti software durante lo sviluppo di una serie di sistemi simili
- Essere consapevoli della modellazione dei processi come modo per descrivere in modo esplicito i processi di sviluppo del software

Per poter valutare i progressi durante lo sviluppo del software, si opta per un approccio graduale con una serie di eventi cardine ben definiti. L'ordinamento lineare delle attività che sta alla base del tradizionale modello di sviluppo del software, il modello a cascata, lo rende un'idealizzazione impossibile della realtà. Presuppone che lo sviluppo del software proceda in modo ordinato e sequenziale. I progetti reali procedono in modi molto meno razionali. Il modello a cascata dello sviluppo del software non è fattibile, proprio come è impraticabile la cascata di Escher, sulla copertina. Questo capitolo discute vari modelli alternativi del processo di sviluppo.

Nel Capitolo 1 abbiamo introdotto un modello semplice del ciclo di vita del software. Abbiamo distinto diverse fasi consecutive: ingegneria dei requisiti, progettazione, implementazione, test e manutenzione. È stato affermato che, in pratica, si utilizzano spesso modelli di processo più sofisticati. In questo capitolo, continuiamo questa discussione. Introduciamo vari modelli alternativi per strutturare il processo di sviluppo del software.

I progetti di sviluppo software sono spesso progetti molto grandi. Molte persone lavorano a lungo su un progetto del genere e quindi l'intero processo deve essere attentamente pianificato e controllato (come visto nel capitolo precedente): i progressi devono essere monitorati, le persone e le risorse devono essere allocate al momento giusto, ecc., è stato sottolineato che l'avanzamento di un progetto di sviluppo software è particolarmente difficile da misurare.

Al fine di controllare i progressi, utilizziamo un processo di sviluppo a fasi in cui vengono stabilite una serie di pietre miliari chiaramente identificabili tra l'inizio e la fine del progetto. Usiamo un meccanismo simile quando costruiamo una casa: si gettano le fondamenta, si raggiunge il primo piano, la casa è resistente alle intemperie e così via. Spesso, i pagamenti rateali sono accoppiati al raggiungimento di tali traguardi.

Molto spesso, le pietre miliari individuate in un progetto di sviluppo software corrispondono ai momenti in cui alcuni **documenti** diventano disponibili:

- dopo l'ingegneria dei requisiti, c'è una **specifica dei requisiti**;
- dopo la fase di progettazione, c'è una **specifica (tecnica) del sistema**;
- dopo l'implementazione, ci sono una serie di **programmi**;
- dopo che il test è stato completato, c'è un **rapporto di prova (callaudo)**.

I modelli tradizionali per lo sviluppo graduale del software sono, in larga misura, "**guidati dai documenti**". La pila di carta che viene prodotta nel corso del progetto guida il processo di sviluppo. Il processo di sviluppo è visto come una serie di trasformazioni. Inizia con un chiaro documento dei requisiti e termina con il codice in esecuzione. Nella prossima sezione, discutiamo il modello a cascata, una nota variazione del modello di processo introdotto nel Capitolo 1. In questa variazione, dopo ogni trasformazione viene eseguito un controllo per determinare se siamo ancora sulla strada giusta.

Questi metodi basati sui documenti sono noti anche come metodi basati *sulla pianificazione* o metodi *pesanti.* Sono guidati dalla pianificazione a causa dell'enfasi su un piano iniziale per l'intero processo e pesanti a causa dell'enfasi posta sul processo.

In molti progetti di sviluppo software, il cambiamento è un dato di fatto. Può anche accadere che il cliente abbia solo una vaga idea dei requisiti del sistema. Negli ultimi anni, sono stati proposti numerosi metodi *agili e leggeri* che presumibilmente si occupano di queste circostanze in rapido cambiamento. Questi metodi raccomandano di non "perdere" tempo in costose attività di pianificazione e progettazione all'inizio, ma di fornire qualcosa di prezioso al cliente il più rapidamente possibile. Sulla base del feedback dell'utente, vengono quindi eseguiti i passaggi successivi. I metodi agili si sono evoluti da approcci, come la prototipazione e lo sviluppo rapido di applicazioni, che cercano di eliminare alcuni o tutti gli svantaggi dell'approccio basato sui documenti sopra menzionato. Discuteremo una serie di approcci leggeri allo sviluppo del software nella Sezione 3.2.

I modelli evolutivi tengono conto del fatto che gran parte di ciò che viene chiamato manutenzione è in realtà evoluzione. Sembrerebbe quindi naturale tenere esplicitamente a mente questa evoluzione anticipata sin dall'inizio. Di solito non è così. Sia nell'approccio di sviluppo pesante che in quello leggero, lo sviluppo iniziale di un sistema software è in generale rigorosamente separato dalla successiva fase di manutenzione. L'obiettivo principale di un progetto di sviluppo software si riduce quindi a fornire all'utente una prima versione del sistema. Ciò potrebbe comportare costi di manutenzione eccessivi in seguito. Per poter valutare correttamente costi e benefici, il nostro obiettivo principale dovrebbe essere il costo totale del ciclo di vita e non solo il costo di sviluppo. Facendo un ulteriore passo avanti, possiamo sostenere che la gestione dovrebbe concentrarsi su insiemi di prodotti simili (noti come "famiglie di prodotti") piuttosto che su singoli prodotti, dando così un incentivo sia alla costruzione di parti riutilizzabili che al riutilizzo di (parti di) esistenti prodotti durante lo sviluppo di nuovi.

Tra tutti i possibili modelli di ciclo di vita, dobbiamo sceglierne uno in particolare per ogni dato progetto. In generale, i metodi pesanti si adattano meglio a progetti (molto) grandi e a

situazioni in cui i requisiti possono essere decisi in una fase iniziale. I metodi leggeri si adattano a situazioni di rapido cambiamento e progetti che non coinvolgono più di, diciamo, 50 persone. Le diverse caratteristiche del progetto e le modalità appropriate per controllarle in modo efficace sono discusse nel Capitolo 8 .

La scelta di un particolare modello del ciclo di vita implica anche la definizione dei singoli passaggi e fasi, la loro possibile interazione, i loro risultati finali, ecc. Utilizzando un linguaggio di modellazione dei processi esplicito (che può essere supportato da strumenti), aumentiamo la nostra comprensione del software processo, ci viene fornita una maniglia per migliorare il nostro controllo sullo sviluppo del software e ci viene fornita una baseline per il miglioramento del processo. Questo tipo di modellazione del processo è discusso nella Sezione 3.7 . A causa dell'enfasi molto maggiore sulla pianificazione, questo tipo di modellazione si adatta meglio ai metodi pesanti.

3.1 IL MODELLO A CASCATA (WATERFALL)

Il modello a cascata è essenzialmente una leggera variazione del modello introdotto nel Capitolo 1. Il modello a cascata è generalmente attribuito a Royce (1970). Tuttavia, nelle pubblicazioni dei primi anni '60 è possibile trovare un approccio chiaramente graduale allo sviluppo del software, inclusi **iterazione** e **feedback**.

Il modello a cascata esprime in particolare l'interazione tra fasi successive. Il test del software non è un'attività che segue rigorosamente la fase di implementazione. In **ogni** fase del processo di sviluppo del software, dobbiamo confrontare i risultati ottenuti con quelli richiesti. In tutte le fasi, la qualità deve essere valutata e controllata.

Nella Figura 3.1 , V&V sta per Verifica e Validazione. La **verifica** chiede se il sistema soddisfa i suoi requisiti (stiamo costruendo il sistema correttamente) e quindi cerca di valutare la correttezza del passaggio alla fase successiva. La **validazione** chiede se il sistema soddisfa i requisiti dell'utente (stiamo costruendo il sistema giusto).

La Figura 3.2 mostra essenzialmente lo stesso modello di processo della Figura 3.1. Per la sua forma, è noto come il modello V. Il modello V mostra come i diversi tipi di test siano correlati alle varie fasi di sviluppo. Ad esempio, durante i test di accettazione, l'intero sistema viene testato rispetto ai requisiti, sotto la supervisione dell'organizzazione dell'utente. Le diverse fasi del test sono discusse nella Sezione 13.9.

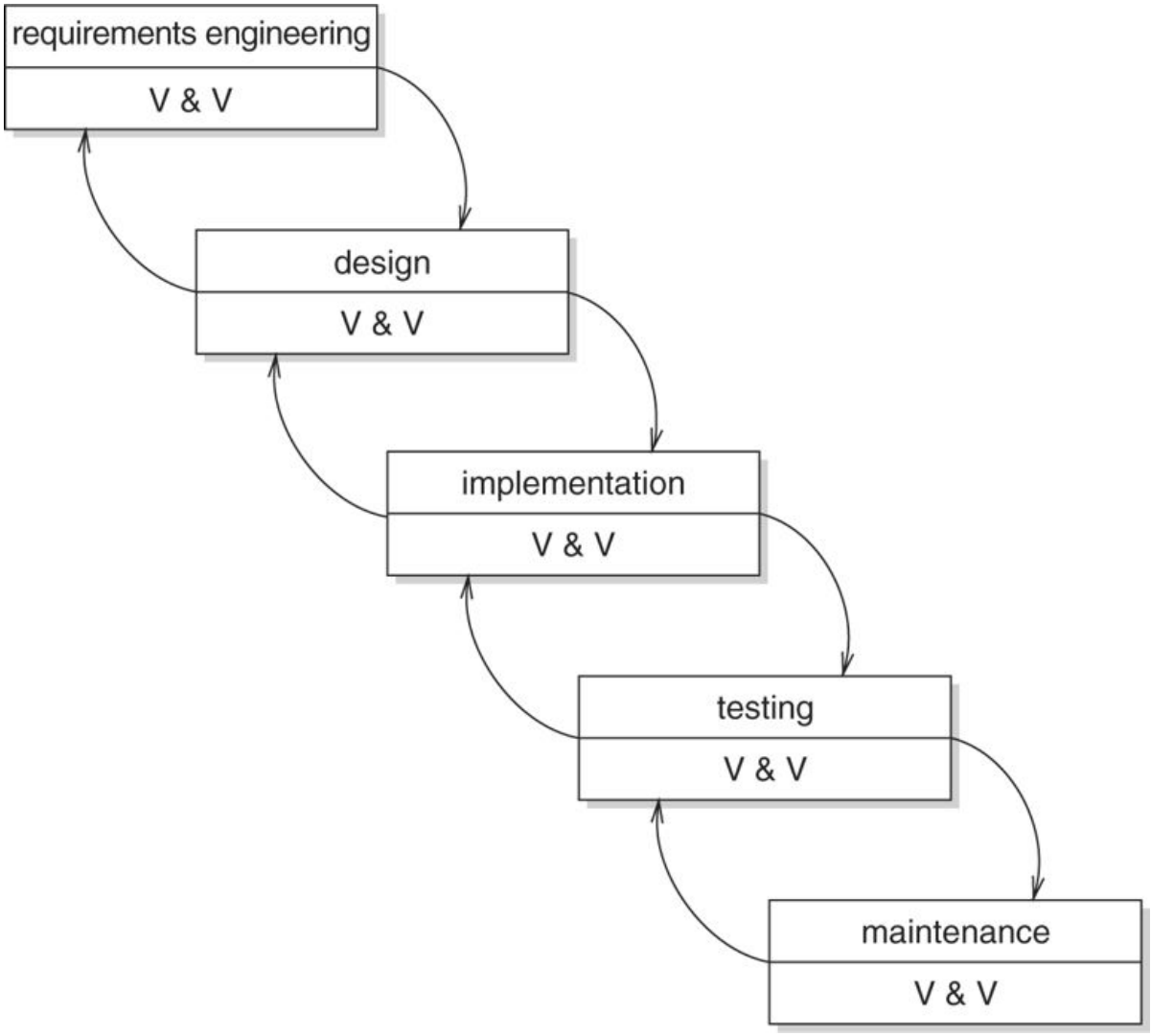


Figura 3.1 Il modello a cascata

Sia il modello introdotto nel **Capitolo 1** che il modello a cascata pongono notevole enfasi su un'analisi attenta prima che il sistema sia effettivamente costruito. Vogliamo evitare di investire troppe energie nella costruzione di un sistema che poi si rivelerà non soddisfare le esigenze dell'utente.

Cerchiamo quindi di identificare e definire le esigenze dell'utente il prima possibile. Questi requisiti sono documentati nella specifica dei requisiti. Sulla base di questo documento potremo verificare in fasi successive se tali requisiti siano rispettati o meno. Poiché nella pratica è difficile, se non impossibile, specificare completamente i requisiti dell'utente, dovrebbe essere effettuato anche un test regolare con il potenziale utente. Questi test sono chiamati validazione. Attraverso le fasi di validazione, possiamo evitare che il sistema in fase di sviluppo diverga dai requisiti dell'utente, eventualmente specificati in modo incompleto.

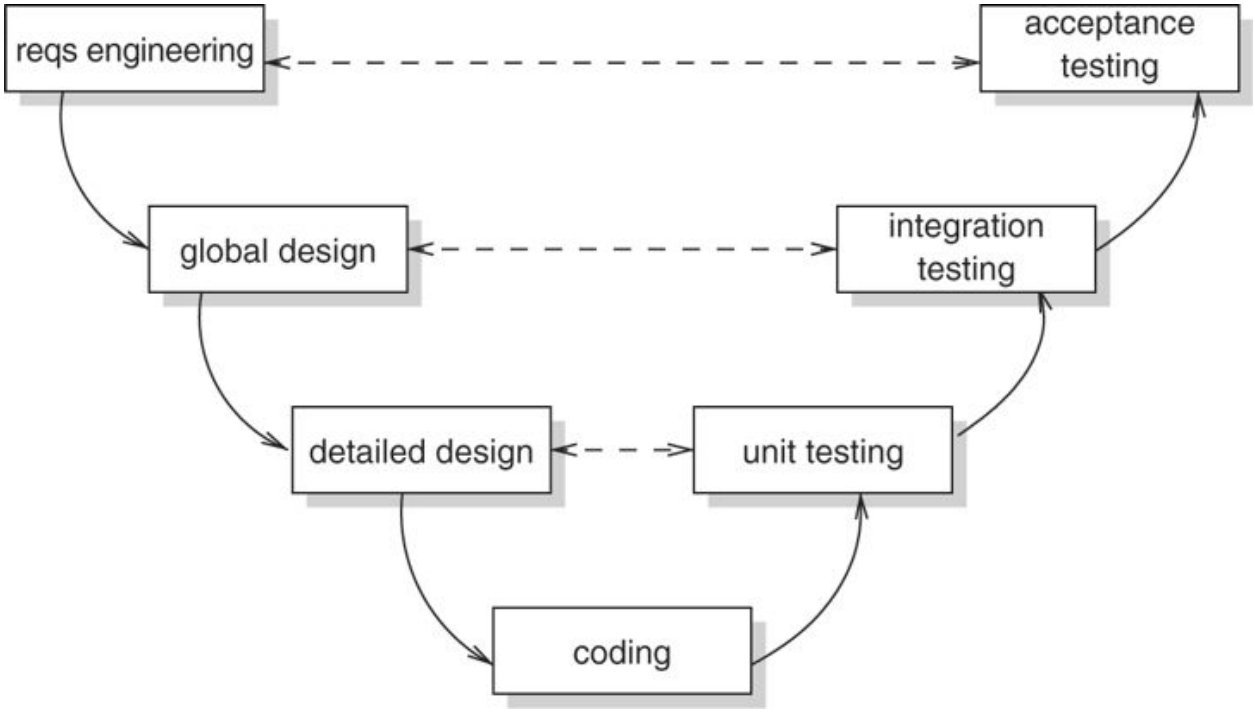


Figura 3.2 Il modello a V

McCracken e Jackson (1981) confrontano il modello a cascata con un negozio dove il cliente è obbligato a dare un ordine entrando. Non c'è possibilità di guardarsi intorno, confrontare i prezzi, cambiare idea o decidere un menu diverso per la cena di oggi. Alcune cose possono essere ordinate per posta, ma non tutte. Il modello a cascata dello sviluppo del software, come quello di Escher, non è realistico.

Tabella 3-1 Ripartizione delle attività nelle fasi del ciclo di vita, dopo (Zelkowitz, 1988)

Attività	**Fase**			
	Design	Codifica	Test d'integrazione	Test di accettazione
Test d'integrazione	4.7	43.4	**26.1**	**25.8**
Codifica	6.9	**70.3**	15.9	6.9
Design	**49.2**	34.1	10.3	6.4

Vi sono ampie prove quantitative che il modello classico basato sui documenti presenta molte carenze. In molti progetti di sviluppo software, la rigida sequenza di fasi raccomandata dal

modello a cascata non viene effettivamente rispettata. La **tabella 3.1** mostra la ripartizione media delle attività nelle fasi del ciclo di vita per un certo numero di progetti. In questo, l'etichetta "codifica" si riferisce a una fase che comprende sia l'implementazione che il test unitario.

Quindi, ad esempio, è stato riscontrato che meno del 50% dello sforzo di progettazione si verifica durante la fase di progettazione effettiva: il 34% dello sforzo di progettazione avviene durante il periodo di codifica e, peggio ancora, oltre il 16% dello sforzo di progettazione avviene dopo il sistema dovrebbe essere finito.

Il comportamento dei singoli progettisti di software può essere caratterizzato come un **processo opportunistico** (Guindon e Curtis, **1988**). I progettisti si spostano avanti e indietro attraverso livelli di astrazione che vanno dai problemi del dominio dell'applicazione ai problemi di codifica. Le date di riferimento sembrano essere alquanto arbitrarie e una parte significativa delle attività attraversa i confini di fase.

3.2 METODI AGILI

L'American Kennel Club afferma: "L'eccitante sport dell'Agilità ha preso d'assalto il mondo. L'anello Agility consente al conduttore e al cane di correre a piena velocità, mentre devono esibirsi in modo accurato e sicuro su telai ad A, passeggiate con cani, altalene e un'ampia varietà di salti e tunnel." Gli ingegneri del software non sono cani, ma l'analogia è chiara.

Quando si utilizza un metodo di sviluppo pesante, è difficile cambiare direzione. Una volta firmato il contratto, il lavoro del team di sviluppo è quello di fornire la funzionalità come stabilito nel contratto. Se la realtà cambia o l'utente ottiene una visione diversa, il cambiamento è difficile da realizzare. Non si adatta all'architettura, richiede rielaborazioni non contabilizzate, allunga i tempi concordati e così via. Una volta avviato, lo sviluppo del software è come un piroscafo oceanico che non cambia facilmente direzione.

Questo fenomeno è stato riconosciuto nel corso degli anni e ne sono seguiti metodi come la prototipazione e lo sviluppo evolutivo. Ma questi metodi portano ancora in qualche modo un sapore ingegneristico con loro. In sostanza, presumono ancora che il mondo sia ordinato. Può essere difficile individuare subito i veri requisiti, ma aumenteranno nel tempo.

I veri metodi agili vedono il mondo come fondamentalmente caotico. Presumono che il cambiamento sia inevitabile. Il loro obiettivo è fornire valore al cliente il più rapidamente possibile e non preoccuparsi di piani e processi estesi che comunque non verranno seguiti. L'essenza dei metodi agili è contenuta nel Manifesto per lo sviluppo agile del software, pubblicato nel 2001 da un gruppo di noti pionieri in questo settore (Beck *et al.*, **2001**). I valori chiave del movimento agile sono:

1. **Gli individui e le interazioni** sono più importanti dei processi e degli strumenti.
2. **Il software funzionante** è più importante della documentazione completa.

3. **La collaborazione con il cliente** è più importante della negoziazione del contratto.
4. **Rispondere al cambiamento** è più importante che seguire un piano.

In inglese:

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

- *Individuals and interactions over processes and tools*
- *Working software over comprehensive documentation*
- *Customer collaboration over contract negotiation*
- *Responding to change over following a plan*

That is, while there is value in the items on the right, we value the items on the left more.

I metodi agili coinvolgono gli utenti in ogni passo compiuto. I cicli di sviluppo sono piccoli e incrementali. La serie di cicli di sviluppo non è ampiamente pianificata in anticipo, ma la nuova situazione viene rivista alla fine di ogni ciclo. Ciò include un po', ma non troppa, pianificazione per il prossimo ciclo.

Alla fine di ogni ciclo, il sistema è attivo e funzionante. Cioè, esiste un sistema funzionante, che offre valore ai suoi utenti. Questo assomiglia molto alla prototipazione evolutiva come discusso nella **Sezione 3.2.1**. Ma la differenza di formulazione riflette un atteggiamento del tutto diverso. Il termine prototipazione suggerisce qualcosa di intermedio, non ancora definitivo, temporaneo. "Codice di lavoro" ha un significato più positivo. Denota qualcosa di immediato valore, anche se non ancora perfetto.

I metodi agili non hanno un'ampia fase architettonica o di progettazione in anticipo. Dopotutto, non ha senso dedicare molti sforzi al design se sai che sarà molto probabilmente una perdita di tempo. È più efficace eseguire il design solo per quanto necessario per il passaggio successivo immediato. I metodi agili hanno spesso un'attività separata, nota come *refactoring,* per migliorare il design dopo ogni incremento.

I metodi agili sono orientati alle persone, piuttosto che ai processi. Sottolineano l'elemento umano nello sviluppo del software. Lo spirito di squadra è considerato molto importante. I rapporti di squadra sono stretti. Spesso, una squadra agile occupa una grande stanza. Anche gli utenti sono sul posto. I metodi agili hanno brevi cicli di comunicazione tra sviluppatori e utenti e tra sviluppatori.

Infine, i metodi agili non spendono molte energie nella documentazione. Perché dedicare tempo a qualcosa che presto sarà obsoleto? Piuttosto, i metodi agili si basano sulla tacita conoscenza delle persone coinvolte. Se hai una domanda, chiedi a uno dei tuoi amici. Non lottare con una grande pila di carta che molto probabilmente non fornirà comunque la risposta.

Alcune persone sostengono che i metodi agili dovrebbero essere "puri" e mostrare tutte le caratteristiche sopra menzionate. Altri ritengono che anche una combinazione di metodi basati sulla pianificazione e agili possa essere produttiva. Siamo d'accordo con quest'ultimo punto di vista. Ad esempio, in ambienti orientati al mercato, dove c'è una forte pressione per fornire funzionalità in modo tempestivo, un'organizzazione può avere una visione a lungo termine della funzionalità e un'architettura ben congegnata, eppure spesso costruire e fornire funzionalità. Cusumano e Yoffe (**1999**) si riferiscono a questo stile di sviluppo come "sincronizzare e stabilizzare". Gli sviluppatori sincronizzano spesso il loro lavoro, spesso alla fine di ogni giornata, costruendo una versione eseguibile del sistema completo e testandolo. Questo è chiamato "costruzione quotidiana". Un po' meno frequentemente, l'insieme delle funzionalità implementate viene stabilizzato, risultando in una nuova baseline da cui ha luogo un ulteriore sviluppo. In questo stile, il valore del "software funzionante" del Manifesto Agile è maggiormente enfatizzato.

Nelle sottosezioni seguenti, discutiamo innanzitutto la **prototipazione e lo sviluppo incrementale,** i primi metodi che riconoscono che un approccio guidato dalla pianificazione spesso non si adatta alla situazione instabile in atto. Lo sviluppo rapido delle applicazioni (RAD) e il metodo di sviluppo dei sistemi dinamici (DSDM) enfatizzano la collaborazione con i clienti e il ruolo delle persone nel processo, e quindi mostrano una serie di caratteristiche chiave dei metodi agili. Infine, discutiamo di programmazione estrema (XP), un metodo agile "puro".

3.2.1 Prototipazione

Spesso è difficile ottenere e mantenere una percezione sufficientemente accurata delle esigenze del potenziale utente. Questo non è sorprendente. In generale, non è sufficiente prendere la situazione *esistente* come l'unico punto di partenza per l'impostazione dei requisiti software. Un motivo importante per intraprendere un progetto di sviluppo software è che non si è soddisfatti della situazione attuale. Ciò che si vuole invece della situazione attuale spesso non è facile da determinare. Ciò vale ancora di più nei casi in cui ci occupiamo di una nuova applicazione e il cliente non conosce tutte le possibilità dell'automazione. In questi casi può essere utile lo sviluppo di uno o più prototipi.

Le analogie con lo sviluppo di altri prodotti sono interessanti qui. Quando si sviluppa una nuova macchina o un chip per computer, si costruiranno anche uno o più prototipi. Questi prototipi vengono testati intensamente prima che venga allestita una vera linea di produzione. Per lo sviluppo del telefono a pulsanti sono stati testati circa 2000 prototipi, con variazioni di forma, dimensioni e posizionamento dei pulsanti, dimensioni e peso del bocchino, ecc.

È possibile seguire una strada simile con lo sviluppo del software. In questo contesto, un prototipo può essere descritto come un modello funzionante di (possibilmente parti di) un sistema software, che enfatizza alcuni aspetti. C'è, tuttavia, una grande differenza tra lo

sviluppo di software e lo sviluppo di prodotti fisici come automobili, chip o telefoni: nello sviluppo di prodotti fisici, i costi più elevati vengono generalmente sostenuti durante la produzione, quando vengono prodotte più copie del prodotto. Nello sviluppo del software, creare più copie del prodotto è quasi gratuito. Se dovessimo seguire l'approccio hardware alla prototipazione nello sviluppo del software e produrre un prototipo con le stesse funzionalità del prodotto finale, svilupperemmo infatti un sistema operativo, con costi altrettanto elevati. Non sembra quindi plausibile ricominciare tutto da capo e sviluppare il sistema 'reale' in modo diverso.

Utilizzando la definizione data sopra e con l'obiettivo di sviluppare un prototipo di software in modo relativamente economico, è importante sottolineare alcuni aspetti. Ciò può essere ottenuto attraverso, ad esempio:

- l'uso di linguaggi di altissimo livello, in cui è possibile creare rapidamente una versione eseguibile (questa versione eseguibile, ma probabilmente piuttosto inefficiente, può essere utilizzata per testare l'usabilità del sistema proposto);
- lo sviluppo di un sistema con meno funzionalità, in particolare per quanto riguarda attributi di qualità come velocità e robustezza.

Una delle principali difficoltà per gli utenti è quella di esprimere con precisione le proprie esigenze. È naturale cercare di chiarirli attraverso la prototipazione. Ciò può essere ottenuto sviluppando rapidamente l'interfaccia utente. Il potenziale utente può quindi lavorare con un sistema che contiene il componente di interazione ma non, o in misura molto minore, il software che effettivamente elabora l'input. In questo modo, l'utente può avere una buona impressione di ciò che il sistema futuro gli fornirà, **prima che** vengano fatti grandi investimenti per realizzare il sistema. La prototipazione diventa così uno strumento per l'ingegneria dei requisiti.

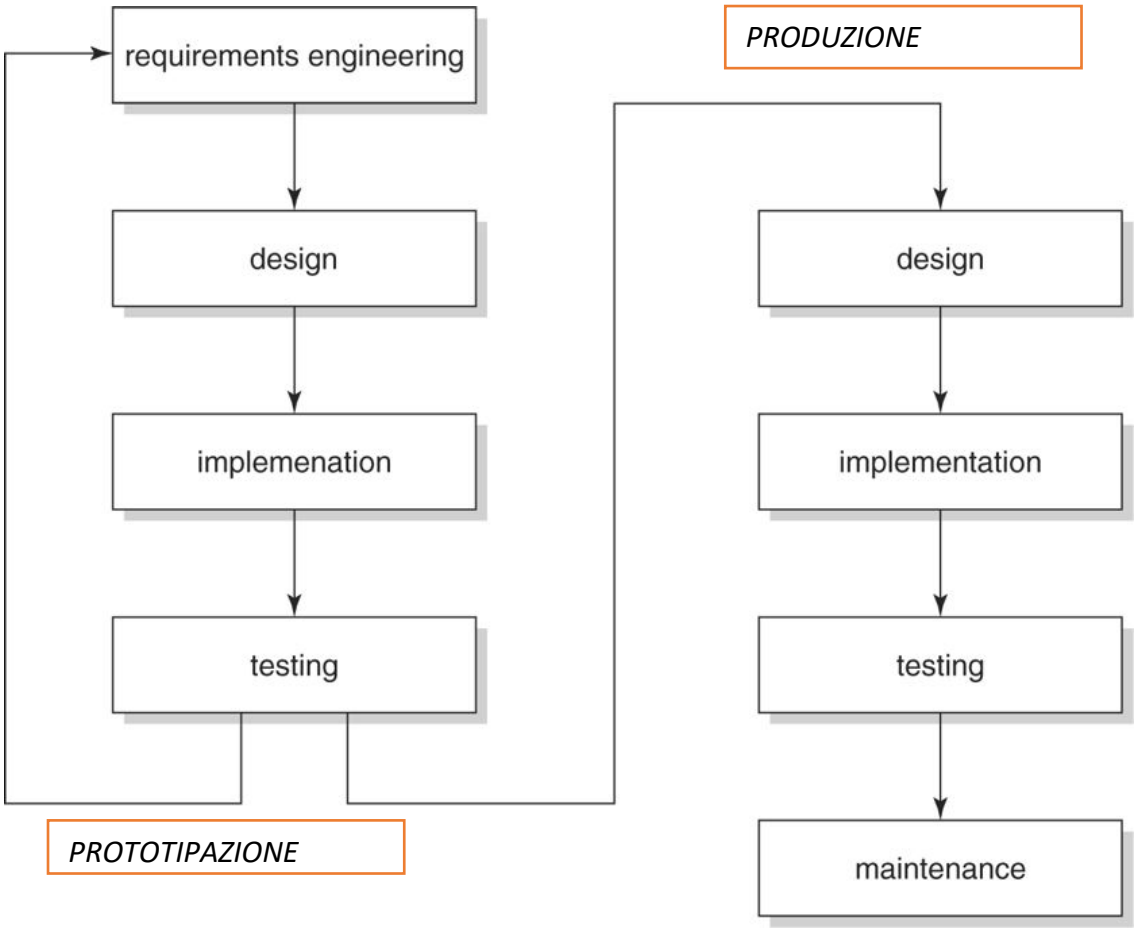


Figure 3.3 Prototyping come strumento per l'ingegneria dei requisiti

Ciò è illustrato graficamente nella **Figura 3.3**, che mostra che le varie fasi vengono attraversate in due modi. Il lato sinistro della figura riguarda le fasi di **prototipazione**. L'iterazione corrisponde al processo di convalida dell'utente, in base al quale requisiti nuovi o modificati attivano il ciclo successivo. Il lato destro riguarda la produzione vera e propria del sistema operativo. La differenza tra i due rami è che, utilizzando tecniche e strumenti diversi, il lato sinistro può essere percorso molto più rapidamente e con costi molto inferiori.

Nella **Figura 3.3**, le fasi di prototipazione e le successive fasi di produzione sono state chiaramente separate. Questo è appropriato, poiché utilizzeremo tecniche diverse durante la fase di produzione effettiva, porremo molta più enfasi sulla documentazione e così via. È anche possibile non trasferire il prodotto software dalle fasi di prototipazione alla fase di produzione vera e propria, ma buttarlo esplicitamente al termine delle fasi di prototipazione. Questo è noto come *prototipazione usa e getta.* Non è necessario farlo, però. Il prototipo può evolvere nel prodotto finale. L'utente inizia con la formulazione dei fabbisogni grezzi, in base ai quali viene prodotta una prima versione del sistema. L'utente inizia a lavorare con questo sistema, che porta a requisiti nuovi o modificati. Viene quindi sviluppata la versione successiva. Dopo una serie di tali iterazioni, l'utente è soddisfatto e l'ultima versione

sviluppata è il prodotto da consegnare. Questo è noto come *prototipazione* ***evolutiva.*** In pratica, la prototipazione evolutiva viene utilizzata molto più spesso della prototipazione usa e getta. Scartare un sistema (parzialmente) funzionante è un ostacolo che non è facile da superare. Nei metodi agili, la frase "codice funzionante" viene spesso utilizzata al posto di "prototipo evolutivo".

Sia la prototipazione usa e getta che quella evolutiva presentano vantaggi e svantaggi. La **tabella 3.2** riassume i pro ei contro emersi da casi di studio che descrivono esperienze reali nell'applicazione di un approccio di prototipazione. Si noti che alcune proprietà possono essere influenzate sia in modo positivo che negativo. A seconda delle circostanze, uno o entrambi possono verificarsi in un progetto reale. Ad esempio, il costo di manutenzione può diminuire perché le esigenze degli utenti sono più soddisfatte o può aumentare perché lo sviluppo è stato svolto in modo rapido e sporco.

Table 3-2 Alcuni pro e contro osservati della prototipazione s

Vantaggi	**Svantaggi**
–Il sistema risultante è più facile da usare	–Il sistema risultante ha più funzioni
–Il sistema risultante ha meno funzioni	–Il design è di qualità inferiore
–Le esigenze degli utenti sono meglio soddisfatte	–Le prestazioni del sistema risultante sono peggiori
–Il design è di qualità superiore	–Il sistema risultante è più difficile da mantenere
–I problemi vengono rilevati prima	–I membri del team dovrebbero essere più esperti
–Il sistema risultante è più facile da mantenere	

Utenti e sviluppatori sono generalmente più positivi sui sistemi sviluppati utilizzando un approccio di prototipazione che sui sistemi sviluppati in un processo a cascata. Questo atteggiamento positivo riguarda sia il processo di sviluppo che il prodotto risultante. Gli utenti si sentono più coinvolti nel processo di sviluppo e hanno meno conflitti con i progettisti. L'ampio coinvolgimento degli utenti si traduce in sistemi che soddisfano meglio le esigenze degli utenti.

Poiché gli utenti non devono esprimere tutte le loro esigenze in anticipo in un approccio di prototipazione, c'è meno tendenza a chiedere campanelli e fischietti (funzionalità extra non

necessarie). Di conseguenza, il risultato finale è un sistema più snello la cui funzionalità corrisponde più strettamente alle reali esigenze dell'utente. Se agli utenti viene mostrato un sistema funzionante in una fase iniziale e viene data l'opportunità di provarlo, è probabile che anche i problemi vengano rilevati in una fase iniziale. Ciò evita uno spreco di manodopera che altrimenti sarebbe necessaria per rifare parte del lavoro. Se gli utenti sono in grado di influenzare e modificare il progetto, le caratteristiche del sistema rifletteranno meglio i loro requisiti e il sistema sarà più facile da usare.

L'uso di strumenti o linguaggi di prototipazione per scopi speciali semplifica l'aggiunta di funzionalità. Poiché l'intervallo di tempo tra le versioni successive del prototipo è ridotto, gli utenti potrebbero pensare che sia facile realizzare nuove funzionalità e specificare requisiti aggiuntivi. Questi aspetti della prototipazione possono comportare sistemi con più funzionalità, anziché meno.

La prototipazione comporta fasi di progettazione iterative e, a causa della ripetuta attenzione al design, la sua qualità può aumentare. Poiché è noto in anticipo che il design si evolverà durante le successive fasi di prototipazione, verrà data maggiore attenzione a fattori di qualità come flessibilità e modularità e, di conseguenza, anche la qualità del progetto potrebbe migliorare. Nella prototipazione usa e getta, la qualità del progetto finale è spesso maggiore a causa dell'esperienza di apprendimento delle fasi di prototipazione. Inoltre, questa fase di progettazione finale è difficilmente, se non del tutto, rattoppata a causa delle azioni di rilavorazione. A causa di questi aspetti, i sistemi risultanti sono spesso più facili da mantenere.

D'altra parte, la prototipazione generalmente non impone rigidi standard di progettazione e sviluppo. Se ci preoccupiamo di un breve tempo di sviluppo, alcune attività necessarie riceveranno meno attenzione. È probabile che la documentazione venga sacrificata per la velocità. A causa delle aggiunte risultanti da frequenti fasi di rilavorazione, la qualità del design di un prototipo evolutivo potrebbe deteriorarsi. Anche per questo motivo, il sistema risultante potrebbe essere meno manutenibile. Soprattutto nei prototipi evolutivi, la robustezza del sistema sarà spesso inferiore a quella consueta con un approccio più tradizionale. Nei metodi agili, il refactoring viene applicato per contrastare questo fenomeno. Infine, le prestazioni tendono a essere peggiori perché l'attenzione è focalizzata sulla funzionalità e le misure delle prestazioni o non vengono prese affatto o vengono prese in un momento in cui sono diventate troppo difficili da realizzare.

Si ritiene generalmente che i progetti di prototipazione richiedano un team esperto. La prototipazione implica prendere decisioni di progettazione di vasta portata, specialmente durante le prime iterazioni. In ogni iterazione, le richieste degli utenti devono essere valutate l'una rispetto all'altra e rispetto alla facilità e al costo della loro realizzazione. I membri del team inesperti hanno maggiori probabilità di fare scelte sbagliate, minacciando così seriamente il successo di uno sforzo di prototipazione.

Da questa discussione, possiamo raccogliere le seguenti raccomandazioni per l'uso delle tecniche di prototipazione:

- la prototipazione è particolarmente utile in situazioni in cui i requisiti dell'utente non sono chiari o ambigui, quando può essere un buon modo per chiarire tali requisiti;
- la prototipazione è inoltre particolarmente utile per i sistemi con una notevole enfasi sull'interfaccia utente e che necessitano di un elevato grado di interazione con l'utente;
- utenti e progettisti devono essere ben consapevoli dell'approccio della prototipazione e delle sue insidie: gli utenti dovrebbero rendersi conto che cambiare il software non è così facile e che un prototipo non è un sistema di produzione-qualità; i progettisti dovrebbero essere consapevoli delle caratteristiche dei progetti di prototipazione e non essere frustrati dai frequenti cambiamenti delle esigenze degli utenti;
- la prototipazione deve essere pianificata e controllata: devono essere imposti limiti al numero di iterazioni; devono esistere procedure esplicite per la documentazione e il collaudo dei prototipi; gli aspetti positivi dell'approccio tradizionale, che rendono il processo gestibile e controllabile, dovrebbero essere applicati alla prototipazione.

Adottando contromisure appropriate, è possibile evitare i potenziali svantaggi della prototipazione. La prototipazione è, quindi, un modello di processo alternativo praticabile per molti progetti di sviluppo software.

3.2.2 Sviluppo incrementale

Nella sezione precedente, abbiamo discusso un modo per utilizzare i prototipi in cui il sistema finale è l'ultimo di una serie di prototipi. Sotto un attento controllo di gestione, al fine di garantire la convergenza, la prossima versione è progettata per soddisfare le nuove o modificate esigenze degli utenti. C'è un altro modo per lavorare verso il sistema finale in una serie di iterazioni: procediamo in **modo incrementale**. La funzionalità del sistema viene prodotta e consegnata al cliente in piccoli incrementi. Partendo dalla situazione esistente, procediamo verso la situazione desiderata in una serie di (piccoli) passi. In ciascuna di queste fasi viene impiegato l'approccio graduale che conosciamo dal modello a cascata.

Sviluppare software in questo modo evita l'effetto 'Big Bang' (in cui non succede nulla per molto tempo e poi, all'improvviso, si verifica una situazione completamente nuova). Invece di creare software, il software cresce. Con questo approccio incrementale, l'utente è strettamente coinvolto nella pianificazione del passaggio successivo. Reindirizzare il progetto diventa più facile poiché potremmo incorporare circostanze modificate più rapidamente.

Lo sviluppo incrementale può essere utilizzato anche per combattere la sindrome da "sovrafunzionalità". Poiché gli utenti hanno difficoltà a formulare le loro reali esigenze, tendono a chiedere troppo. Non avendo la necessaria conoscenza della malleabilità del software e del suo processo di sviluppo, possono essere inclini a pensare che tutto possa essere realizzato. Di conseguenza, le caratteristiche essenziali compaiono accanto a campanelli e fischietti nell'elenco dei requisiti. Gli analisti non sono in grado di distinguere l'uno dall'altro, né di stimare con precisione lo sforzo richiesto per implementare le singole funzionalità. È probabile, quindi, che molti sforzi vengano spesi per realizzare funzionalità che non sono realmente necessarie. Di conseguenza, molti dei sistemi odierni offrono funzionalità

avanzate ma, allo stesso tempo, non sono adatti per l'attività da svolgere. Per prima cosa, questi sistemi sono difficili da usare semplicemente a causa della complessità sostenuta dalla loro ricca funzionalità.

Con l'approccio incrementale, l'attenzione viene prima focalizzata sulle caratteristiche essenziali. Funzionalità aggiuntive sono incluse solo se e quando è necessario. I sistemi così sviluppati tendono ad essere più snelli e tuttavia forniscono un supporto sufficiente ai loro utenti. Con l'approccio incrementale, spesso vengono affrontate per prime le parti più difficili o le parti che presentano il rischio più elevato rispetto al completamento con successo del progetto.

Seguendo questa linea di pensiero, Boehm (**1988**) propone un **modello a spirale** del processo di sviluppo del software, in cui ogni rivoluzione della spirale dà origine alle seguenti attività:

1. **Fase 1: Definizione di obiettivi e alternative, nonché descrizione delle condizioni**

   Un ciclo tipico nel modello a spirale inizia con la valutazione di quali obiettivi devono essere associati ai singoli passaggi dello sviluppo del software. Può trattarsi, ad esempio, del miglioramento delle prestazioni o dello sviluppo di funzionalità. Allo stesso tempo, è importante definire alternative per l'implementazione (ad esempio progetto A vs progetto B) e determinare le condizioni generali, i costi o il dispendio in termini di tempo.

2. **Fase 2: Valutazione delle alternative**

   Il passaggio successivo consiste nella valutazione delle alternative: in questa fase la definizione degli obiettivi e le condizioni fungono da punti di riferimento cruciali. In questa fase del ciclo del modello a spirale, vengono identificati i dubbi che rappresentano un rischio significativo per l'avanzamento del progetto di software. Si passa quindi allo sviluppo della strategia più a basso rischio e più economica, utilizzando metodi come prototipazione, simulazioni, test di riferimento, modelli di analisi e sondaggi tra gli utenti.

3. **Fase 3: Sviluppo e controllo dello stato intermedio**

   Dopo l'analisi dei rischi, si prosegue con l'effettivo sviluppo del software, sebbene questa fase sia sempre caratterizzata dai relativi rischi residui. Se, ad esempio, i rischi relativi alle prestazioni o all'interfaccia utente o al controllo dell'interfaccia interna hanno un peso molto importante sul processo di sviluppo, l'opzione da prediligere è una strategia di sviluppo evolutiva che specifichi il progetto in modo più preciso e ottimizzi i prototipi. Il codice effettivo viene scritto e testato più volte fino a raggiungere il risultato desiderato, che serve quindi come base a basso rischio per le ulteriori fasi di sviluppo.

4. **Fase 4: Pianificazione del ciclo successivo**

   Concluso un ciclo, inizia già la pianificazione del ciclo successivo. Da un lato, se l'obiettivo di un ciclo è stato raggiunto, può trattarsi della normale continuazione del progetto con il passaggio all'obiettivo successivo. Dall'altro, se il precedente passaggio di sviluppo fosse risultato difettoso, può anche trattarsi della ricerca di soluzioni. Così, ad esempio, la strategia

precedente può essere sostituita da una delle alternative definite precedentemente o da una nuova alternativa. A questo punto è quindi possibile fare un nuovo tentativo per raggiungere l'obiettivo specificato.identificare il sottoproblema che presenta il rischio associato più elevato;

I vari modelli di processo già discussi possono essere accoppiati con il modello a spirale di Boehm in modo naturale (vedi **Figura 3.4**):

- Se l'ottenimento della corretta serie di requisiti dell'utente è considerata l'area a più alto rischio, seguire la spirale alcune volte per risolvere questo sottoproblema (ad esempio, il prototipo).
- Se il problema principale è ottenere un sistema robusto e ben documentato da una precisa specifica dei requisiti, seguire la spirale una volta, utilizzando il modello di processo tradizionale con le sue fasi e le tappe intermedie corrispondenti come passaggi intermedi.
- Se si sviluppa software in modo incrementale, tenere traccia della spirale un certo numero di volte, una per ogni incremento.
- Durante la manutenzione, gli errori segnalati o la modifica dei requisiti sono fattori scatenanti per tenere traccia della spirale.

Visto in questo modo, il modello a spirale sussume gli altri modelli di processo discussi finora.

Lo sviluppo incrementale è fortemente sostenuto da Gilb (**1988**). È dubbio che l'incremento temporale auspicato da Gilb, fino a un massimo di qualche settimana, sia sempre ragionevole. Ma i vantaggi dello sviluppo incrementale sono notevoli, anche con incrementi temporali diversi. Le sorprese che si nascondono all'interno dell'approccio tradizionale e che pongono notevoli difficoltà sul lato gestionale dei progetti di sviluppo software possono essere notevolmente ridotte quando il software viene sviluppato e distribuito in modo incrementale.

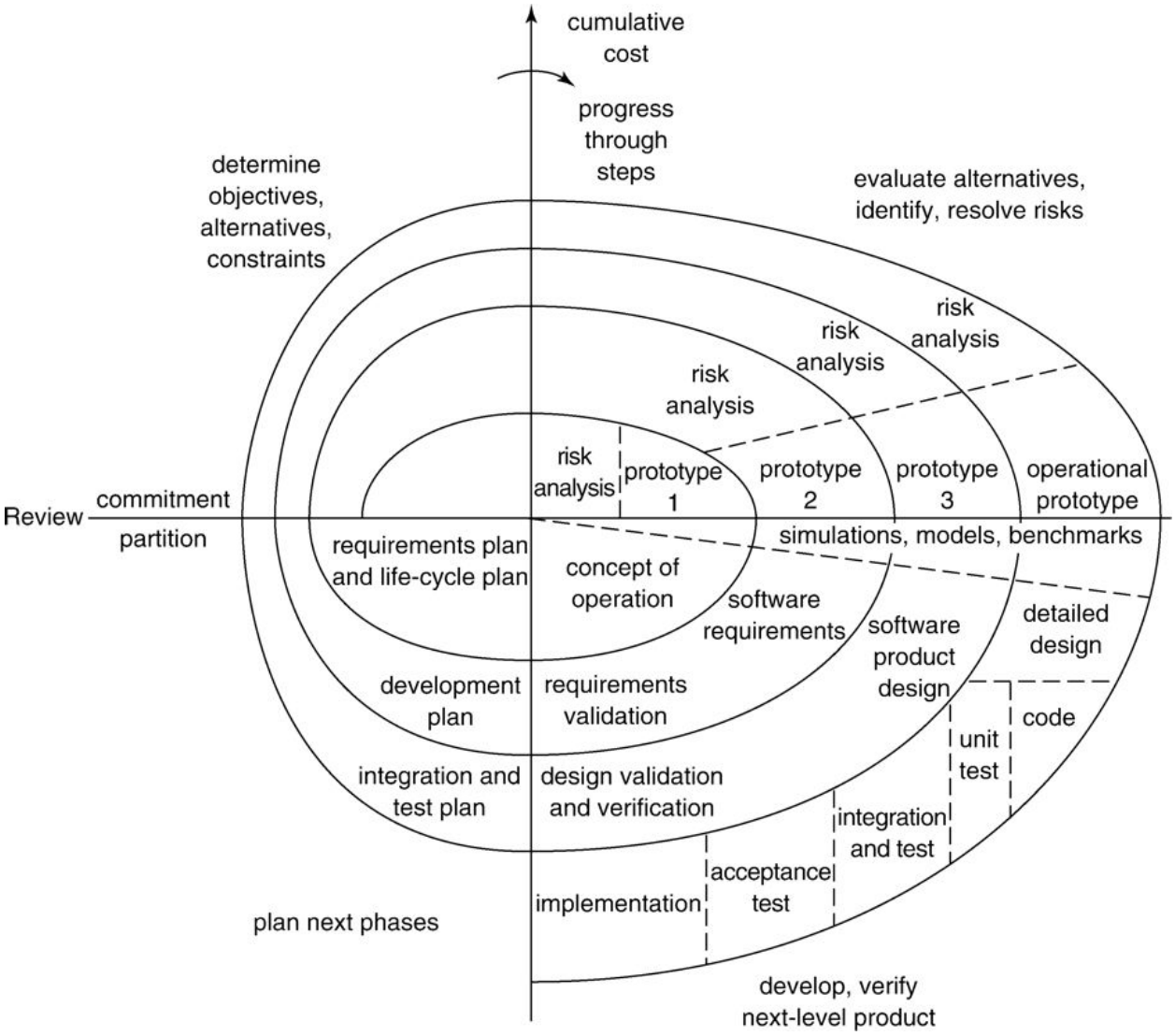


Figure 3.4 Il modello a spirale (Fonte: BW Boehm, Un modello a spirale di sviluppo e miglioramento del software, IEEE Computer 21:5, © 1988 IEEE. Riprodotto con autorizzazione.)

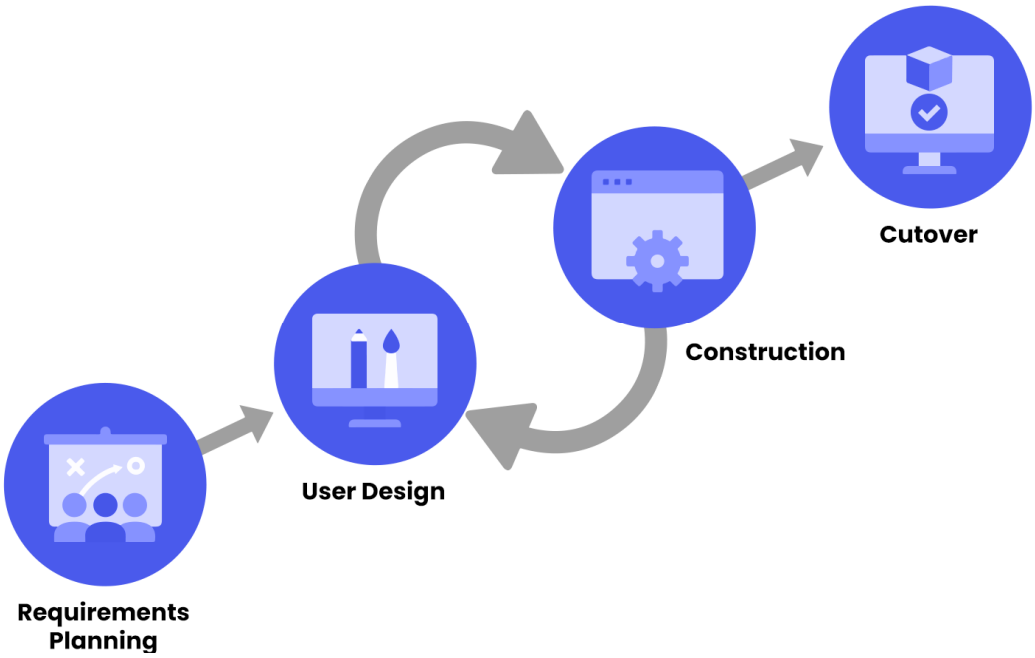
3.2.3 RAD - Metodo di sviluppo rapido di applicazioni e sistemi dinamici

Il Rapid Application Development (RAD) ha molto in comune con altri modelli di processo di sviluppo iterativo. Sottolinea il coinvolgimento degli utenti, la prototipazione, il riutilizzo, l'uso di strumenti automatizzati e piccoli team di sviluppo. In aggiunta a ciò, utilizza la nozione di *time box,* un intervallo di tempo fisso entro il quale vengono svolte le attività. Nella maggior parte dei modelli di sviluppo, viene fissata una serie di requisiti e quindi il progetto tenta di soddisfare tali requisiti entro un periodo di tempo stimato. All'interno di RAD, prima viene deciso il periodo di tempo e poi il progetto cerca di realizzare la funzionalità richiesta entro quel lasso di tempo. Se si scopre che non tutte le funzionalità possono essere realizzate entro

l'intervallo di tempo, alcune delle funzionalità vengono sacrificate. Il termine concordato è comunque inamovibile.

Il ciclo di vita della RAD si compone di quattro fasi:

- pianificazione dei requisiti,
- progettazione dell'applicazione,
- costruzione,
- taglio.



Le fasi di pianificazione dei requisiti e di progettazione dell'applicazione hanno molto in comune e possono essere combinate per progetti più piccoli. Insieme, in genere impiegano meno di due mesi. Le principali tecniche utilizzate in queste fasi sono note come Joint Requirements Planning (JRP) e Joint Application Design (JAD). Entrambe queste tecniche fanno largo uso di workshop in cui gli sviluppatori ei potenziali utenti lavorano *insieme* (da cui l'aggettivo *joint)*.

L'obiettivo del workshop JRP è quello di ottenere i requisiti giusti la prima volta. Per questo motivo, è fondamentale che siano presenti gli attori chiave, ovvero gli utenti finali del sistema. I requisiti hanno la priorità, poiché è probabile che non tutti verranno implementati nella prima versione del sistema. Questa definizione delle priorità dei requisiti è nota come *triage.* Il triage di solito significa un processo utilizzato sul campo di battaglia e nei pronto soccorso per smistare i feriti in gruppi in base alla loro necessità o al probabile beneficio di cure mediche immediate. In RAD, il processo di triage viene utilizzato per assicurarsi che i requisiti più importanti vengano soddisfatti per primi. Il risultato di questo processo è spesso una priorità indicata dall'acronimo MoSCoW:

- **Must have** sono requisiti assolutamente necessari.
- **Should have** sono requisiti importanti, ma non assolutamente necessari per un sistema utilizzabile.
- **Could have** sono requisiti che vengono implementati solo se il tempo lo consente.
- **Won't have** sono requisiti non richiesti che rimarranno per la prossima iterazione.

Ad esempio, si consideri lo sviluppo di un Sistema Informativo Bibliotecario. La categoria **Must have** includerebbe la possibilità di prendere in prestito e restituire un articolo e di iscriversi come membro. La categoria **Should have** potrebbe includere strutture per prenotare un articolo. La categoria **Could have** potrebbe includere la possibilità di gestire le multe per gli articoli restituiti in ritardo. Infine, la categoria **Won't have** nella prima iterazione potrebbe includere funzioni per profilare gli utenti e notificare loro gli articoli appena arrivati.

È consuetudine avere due workshop JAD durante la fase di progettazione. Anche in questo caso, gli utenti finali svolgono un ruolo essenziale in questi workshop. Il primo workshop JAD fornisce un progetto iniziale del sistema. Gli sviluppatori costruiscono quindi un prototipo, che deve essere sperimentato dagli utenti. Questo prototipo viene valutato durante il secondo workshop JAD, vengono decisi i miglioramenti e il progetto viene finalizzato.

Il sistema è costruito da un team altamente qualificato di circa quattro persone, un team "Skilled With Advanced Tools" (SWAT) (vedi anche **Sezione 5.2**). Il team SWAT viene coinvolto dopo il primo workshop JAD. Il team in genere fa il suo lavoro in meno di due mesi. Per poterlo fare, si fa un uso massiccio di strumenti e i componenti esistenti vengono riutilizzati quando possibile. Entro il tempo assegnato (il time box), il team SWAT costruisce una serie di prototipi evolutivi. Gli sviluppatori e gli utenti lavorano a stretto contatto durante questo processo. Ogni prototipo viene esaminato dagli utenti e le sessioni di revisione generano richieste di funzionalità migliorate o modificate. Il termine concordato *non viene* superato. Se necessario, alcune delle funzionalità vengono invece sacrificate.

Affinché un team SWAT operi con successo e fornisca un buon risultato in un arco di tempo molto breve, deve sentire una definita "proprietà" del problema da affrontare. In una situazione del genere, non è molto utile se i tempi e le scadenze sono fissati da qualche manager. Invece, il team SWAT stesso stima il tempo, il team SWAT decide il numero e la durata delle caselle temporali e il team SWAT decide quale funzionalità implementare in ogni iterazione.

Durante la fase di cutover, avviene il collaudo finale del sistema, gli utenti vengono formati e il sistema viene installato.

Ci sono molte variazioni sul modello di processo RAD descritto sopra. Ad esempio, è possibile avere time box espliciti per la costruzione di ciascuno dei prototipi intermedi. È anche possibile avere sessioni JRP o JAD dopo ogni ciclo di prototipazione. Gli ingredienti principali, tuttavia, rimangono: prototipazione, notevole coinvolgimento degli utenti, team SWAT e time box.

JRP e JAD hanno molto in comune con un metodo di progettazione noto come design partecipativo (PD) o la scuola scandinava di sviluppo software. Entrambi enfatizzano il coinvolgimento dell'utente finale ma differiscono, tuttavia, nei loro obiettivi. Il coinvolgimento degli utenti in JRP e JAD ha principalmente lo scopo di accelerare il processo di produzione del sistema giusto. Il coinvolgimento degli utenti nel PD è motivato da un forte interesse per il contesto sociale dell'ambiente di lavoro.

Il metodo di sviluppo di sistemi dinamici (DSDM) è un framework ben noto che si basa su RAD. DSDM si basa sui nove principi illustrati nella **Tabella 3.3**. DSDM è un framework senza scopo di lucro gestito dal Consorzio DSDM (vedi **www.dsdm.org**). Una descrizione di alto livello del framework è data in (Stapleton, **2003**). L'insieme completo delle pratiche DSDM è disponibile solo per i membri del Consorzio DSDM.

Il processo DSDM ha cinque fasi (vedi **Figura 3.5**):

- Nello studio di **fattibilità** viene valutata l'idoneità del DSDM per il progetto in corso. Questo è diverso da uno studio di fattibilità più tradizionale, in cui l'enfasi è sulla fattibilità di una soluzione. Accanto a domande come "Possiamo costruire questo sistema?", la domanda "Il DSDM è appropriato per questo progetto?" bisogna anche rispondere. Le caratteristiche che rendono il DSDM un approccio fattibile riflettono i principi del metodo: deve essere possibile identificare gli utenti del sistema, il sistema non deve essere troppo grande e non tutti i requisiti devono essere noti in anticipo.

Tabella 3-3I principi del DSDM

Principio	Descrizione
Il coinvolgimento attivo dell'utente è fondamentale.	Gli utenti supportano il team durante tutto il progetto, non solo durante l'elicitazione dei requisiti e i test di accettazione. Il canale di comunicazione tra il team di sviluppo e gli utenti è breve.
La squadra deve avere il potere di prendere decisioni.	La squadra deve essere in grado di prendere decisioni rapide. Lo slancio viene perso se la squadra deve attendere l'approvazione esterna di ogni piccola decisione.
L'attenzione è rivolta alla consegna frequente dei prodotti.	La consegna frequente consente un feedback frequente da parte della comunità degli utenti e il controllo del processo decisionale da parte dei manager.

Principio	Descrizione
L'idoneità allo scopo commerciale è il criterio essenziale per l'accettazione dei risultati finali.	L'enfasi è sulla fornitura del prodotto giusto, non sulla doratura o sulla conformità alle specifiche.
Lo sviluppo iterativo e incrementale è necessario per convergere su una soluzione aziendale accurata.	I requisiti non possono essere determinati completamente in anticipo. I sistemi devono evolversi e rielaborarli è un dato di fatto.
I cambiamenti durante lo sviluppo sono reversibili.	Potrebbe essere preso un percorso sbagliato ed è necessario tornare indietro per raggiungere di nuovo un punto sicuro.
I requisiti sono di base a un livello elevato.	I requisiti di alto livello vengono determinati durante la fase di studio aziendale e i requisiti dettagliati vengono determinati durante le fasi iterative successive.
I test sono integrati durante tutto il ciclo di vita.	Il test non viene posticipato fino al termine della codifica. Viene eseguito in modo incrementale, dopo che ogni componente è stato scritto.
È essenziale un approccio collaborativo e cooperativo tra le parti interessate.	Le responsabilità sono condivise e gli sviluppatori hanno bisogno del supporto degli utenti finali per decidere cosa sviluppare.

- Lo **studio aziendale** si traduce in una descrizione di alto livello dei processi aziendali rilevanti per il sistema. Questi sono determinati utilizzando workshop facilitati (come JRP) e si traducono in una baseline di alto livello. In questa fase viene determinata anche l'architettura di alto livello.
- L'**iterazione del modello funzionale si** traduce in modelli di analisi, prototipi e implementazione dei principali componenti funzionali. L'iterazione viene eseguita in

caselle temporali in genere da due a sei settimane. Ogni iterazione consiste in quattro attività: identifica cosa farai; concordare come lo farai; fallo; e controlla di averlo fatto.

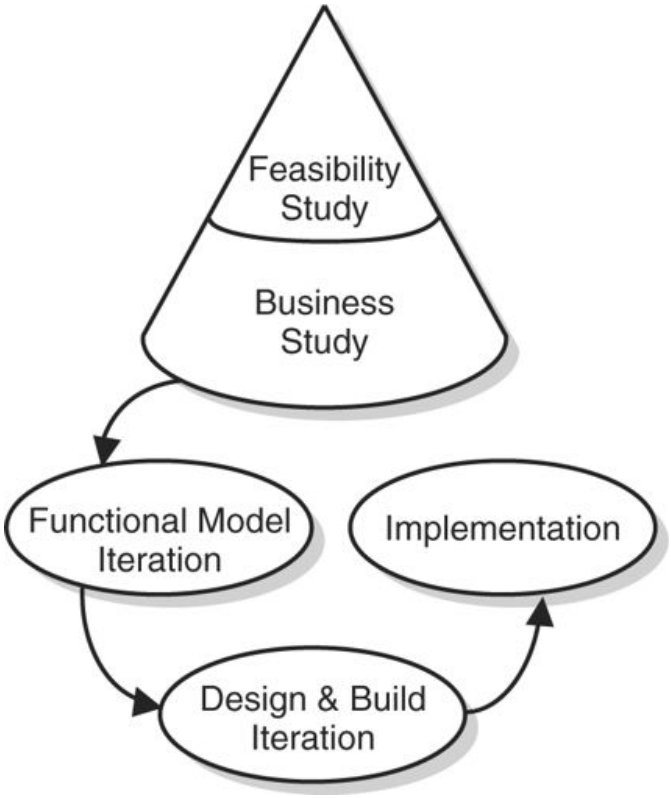


Figure 3.5 Il processo DSDM

- Durante la **progettazione e l'iterazione di costruzione,** il sistema viene ingegnerizzato secondo uno standard sufficientemente elevato. Anche in questo caso, il lavoro viene svolto in caselle temporali generalmente da due a sei settimane e vengono eseguite le stesse quattro attività dell'iterazione del modello funzionale. Sebbene l'enfasi delle iterazioni del modello funzionale sia sulla decisione di *cosa* costruire, e quella delle iterazioni di progettazione e costruzione sia sulla costruzione di una soluzione adeguatamente ingegnerizzata, la distinzione tra questi due tipi di iterazione non è sempre netta.
- Nella fase di **implementazione,** il sistema viene trasferito all'ambiente del cliente. Questa fase include anche la formazione degli utenti.

3.2.4 Programmazione estrema - eXtreme Programming

La programmazione estrema (XP in breve) è un metodo agile puro. XP si basa su una serie di best practices conosciute da molto tempo. XP porta queste pratiche a livelli estremi.

Ad esempio, sappiamo che la lettura del codice da parte dei colleghi, come avviene nelle procedure dettagliate e nelle ispezioni del codice (vedere anche **il Capitolo 13**) è un metodo di test molto efficace. In XP, lo si fa sempre: due programmatori lavorano insieme sullo schermo di un computer. Uno di loro fa la codifica, l'altro si guarda alle spalle, dà consigli,

nota piccoli errori, fa domande e così via. Agiscono come pilota e copilota. In qualsiasi momento, i ruoli possono cambiare. Questa pratica è chiamata *programmazione a coppie.*

L'insieme completo delle pratiche di XP è riportato nella **Tabella 3.4** . In genere, una squadra di XP non è troppo grande e occupa una stanza. Le riunioni di pianificazione sono molto brevi e coinvolgono l'immediata funzionalità da consegnare al cliente. La pianificazione include sia il cliente che il personale tecnico. Il cliente deve stabilire le priorità, determinare le date di rilascio e così via. Il cliente descrive le caratteristiche desiderabili del sistema nelle storie e le registra su schede. Le persone tecniche stimano quanto tempo ci vorrà per implementare una storia e decidono l'ordine in cui verranno implementate le storie all'interno di una versione.

Tabella 3.4 Pratiche di programmazione estrema

1. Intera squadra (whole team)

Tutti i collaboratori di un progetto XP siedono insieme, membri di un team. Questo team deve *includere un rappresentante aziendale*, il "Cliente", che fornisce i requisiti, stabilisce le priorità e guida il progetto. È meglio se il Cliente o uno dei suoi assistenti è un vero utente finale che conosce il dominio e ciò di cui ha bisogno. Il team avrà ovviamente dei programmatori. Il team può includere dei tester, che aiutano il Cliente a definire i test di accettazione del cliente. Gli analisti possono fungere da assistenti del Cliente, aiutando a definire i requisiti. Di solito c'è un coach, che aiuta il team a rimanere in carreggiata e facilita il processo. Potrebbe esserci un manager, che fornisce risorse, gestisce la comunicazione esterna, coordina le attività. Nessuno di questi ruoli è necessariamente di esclusiva proprietà di un solo individuo: tutti in un team XP contribuiscono in qualsiasi modo possibile. I team migliori non hanno specialisti, solo collaboratori generici con competenze speciali.

2. Gioco di pianificazione

La pianificazione XP affronta due questioni chiave nello sviluppo software: prevedere cosa verrà realizzato entro la data di scadenza e determinare cosa fare dopo. L'enfasi è sulla gestione del progetto, il che è piuttosto semplice, piuttosto che sulla previsione esatta di cosa sarà necessario e di quanto tempo ci vorrà, il che è piuttosto difficile. Ci sono due passaggi chiave di pianificazione in XP, che affrontano queste due questioni: La pianificazione delle release è una pratica in cui il cliente presenta le funzionalità desiderate ai programmatori e i programmatori ne stimano la difficoltà. Con le stime dei costi in mano e con la conoscenza dell'importanza delle funzionalità, il cliente stabilisce un piano per il progetto. I piani di release iniziali sono necessariamente imprecisi: né le priorità né le stime sono veramente solide e finché il team non inizia a lavorare, non sapremo quanto velocemente andranno. Tuttavia, anche il primo piano di release è abbastanza accurato per il processo decisionale e i team XP rivedono regolarmente il piano di release. La pianificazione delle iterazioni è la pratica in base alla quale il team riceve indicazioni ogni due settimane. I team XP sviluppano software in "iterazioni" di due settimane, consegnando software utile e funzionante alla fine di ogni iterazione. Durante la pianificazione delle iterazioni, il cliente presenta le funzionalità

desiderate per le due settimane successive. I programmatori le suddividono in attività e ne stimano i costi (a un livello di dettaglio più fine rispetto alla pianificazione delle release). In base alla quantità di lavoro svolto nell'iterazione precedente, il team si iscrive a ciò che verrà intrapreso nell'iterazione corrente. Questi passaggi di pianificazione sono molto semplici, ma forniscono informazioni molto buone e un controllo di guida eccellente nelle mani del Cliente. Ogni due settimane, la quantità di progressi è completamente visibile. Non esiste un "novanta percento fatto" in XP: una storia di funzionalità è stata completata o non lo è stata.

3. Test dei clienti

Come parte della presentazione di ogni funzionalità desiderata, il cliente XP definisce uno o più test di accettazione automatizzati per dimostrare che la funzionalità funziona. Il team crea questi test e li usa per dimostrare a se stesso e al cliente che la funzionalità è implementata correttamente. L'automazione è importante perché nella fretta, i test manuali vengono saltati. È come spegnere le luci quando la notte diventa più buia. I migliori team XP trattano i test dei clienti nello stesso modo in cui trattano i test dei programmatori: una volta eseguito il test, il team lo mantiene in esecuzione correttamente in seguito. Ciò significa che il sistema non fa che migliorare, avanzando sempre, senza mai arretrare.

4. Piccole uscite (small releases)

I team XP praticano le piccole release in due modi importanti: Innanzitutto, il team rilascia software funzionante e testato, che fornisce il valore aziendale scelto dal Cliente, a ogni iterazione. Il Cliente può utilizzare questo software per qualsiasi scopo, che si tratti di valutazione o persino di rilascio agli utenti finali (altamente consigliato). L'aspetto più importante è che il software sia visibile e fornito al cliente alla fine di ogni iterazione. Ciò mantiene tutto aperto e tangibile. In secondo luogo, i team XP rilasciano frequentemente anche ai loro utenti finali. I progetti Web XP rilasciano con una frequenza giornaliera, i progetti interni mensilmente o più frequentemente. Per maggiori informazioni, vedere Continuous Integration e notare che queste release frequenti sono mantenute affidabili dall'ossessione di XP per i test, come descritto qui in Customer Tests and Test-Driven Development.

5. Design semplice

I team XP creano software con un design semplice ma sempre adeguato. Iniziano in modo semplice e, tramite test di programmazione e miglioramento del design, lo mantengono tale. Un team XP mantiene il design esattamente adatto alla funzionalità corrente del sistema. Non c'è movimento sprecato e il software è sempre pronto per il futuro. Il design in XP non è una cosa una tantum o una cosa iniziale, è una cosa continua. Ci sono fasi di design nella pianificazione del rilascio e nella pianificazione dell'iterazione, inoltre i team si impegnano in rapide sessioni di design e revisioni del design tramite refactoring, nel corso dell'intero progetto. In un processo incrementale e iterativo come Extreme Programming, un buon design è essenziale. Ecco perché c'è così tanta attenzione al design nel corso dell'intero sviluppo.

6. Programmazione in coppia

Tutto il software di produzione in XP è realizzato da due programmatori, seduti fianco a fianco, sulla stessa macchina. Questa pratica assicura che tutto il codice di produzione venga esaminato da almeno un altro programmatore e si traduce in una progettazione migliore, test migliori e codice migliore. Potrebbe sembrare inefficiente avere due programmatori che svolgono "il lavoro di un programmatore", ma è vero il contrario. La ricerca sulla programmazione in coppia mostra che l'accoppiamento produce codice migliore più o meno nello stesso tempo di programmatori che lavorano singolarmente. Proprio così: due teste sono davvero meglio di una! Alcuni programmatori si oppongono alla programmazione in coppia senza averla mai provata. Ci vuole un po' di pratica per farla bene, e devi farla bene per qualche settimana per vedere i risultati. Il novanta percento dei programmatori che imparano la programmazione in coppia la preferisce, quindi la consigliamo vivamente a tutti i team. L'accoppiamento, oltre a fornire codice e test migliori, serve anche a comunicare la conoscenza all'interno del team. Quando i pair cambiano, tutti ottengono i benefici della conoscenza specialistica di tutti. I programmatori imparano, le loro competenze migliorano, diventano più preziosi per il team e per l'azienda. L'accoppiamento, anche da solo al di fuori di XP, è una grande vittoria per tutti.

7. Sviluppo basato sui test

Extreme Programming è ossessionata dal feedback e, nello sviluppo software, un buon feedback richiede un buon testing. I migliori team XP praticano lo "sviluppo guidato dai test", lavorando in cicli molto brevi di aggiunta di un test e poi facendolo funzionare. Quasi senza sforzo, i team producono codice con una copertura di test di quasi il 100 percento, il che è un grande passo avanti nella maggior parte delle aziende. (Se i tuoi programmatori stanno già eseguendo test ancora più sofisticati, più potere a te. Continua così, può solo aiutare!) Non basta scrivere i test: bisogna eseguirli. Anche qui, Extreme Programming è estremo. Questi "test del programmatore", o "test unitari" sono tutti raccolti insieme, e ogni volta che un programmatore rilascia del codice nel repository (e le coppie in genere rilasciano due volte al giorno o più), ogni singolo test del programmatore deve essere eseguito correttamente. Al cento per cento, sempre! Ciò significa che i programmatori ricevono un feedback immediato su come stanno procedendo. Inoltre, questi test forniscono un supporto inestimabile man mano che la progettazione del software viene migliorata.

8. Miglioramento del design (refactoring)

Extreme Programming si concentra sulla fornitura di valore aziendale in ogni iterazione. Per raggiungere questo obiettivo nel corso dell'intero progetto, il software deve essere ben progettato. L'alternativa sarebbe rallentare e alla fine bloccarsi. Quindi XP utilizza un processo di miglioramento continuo del design chiamato Refactoring, dal titolo del libro di Martin Fowler, " Refactoring: Improving the Design of Existing Code". Il processo di refactoring si concentra sulla rimozione della duplicazione (un segno sicuro di cattiva progettazione) e sull'aumento della "coesione" del codice, riducendo al contempo l'"accoppiamento".

Un'elevata coesione e un basso accoppiamento sono stati riconosciuti come i tratti distintivi di un codice ben progettato per almeno trent'anni. Il risultato è che i team XP iniziano con un buon design semplice e hanno sempre un buon design semplice per il software. Ciò consente loro di sostenere la velocità di sviluppo e, di fatto, di aumentare generalmente la velocità man mano che il progetto procede. Il refactoring è, ovviamente, fortemente supportato da test completi per essere certi che, man mano che il design evolve, nulla si rompa. Quindi i test dei clienti e i test dei programmatori sono un fattore abilitante critico. Le pratiche XP si supportano a vicenda: sono più forti insieme che separatamente.

9. Integrazione continua

I team di Extreme Programming mantengono il sistema completamente integrato in ogni momento. Diciamo che le build giornaliere sono per i deboli: i team XP creano build più volte al giorno. (Un team XP di quaranta persone crea build almeno otto o dieci volte al giorno!) Il vantaggio di questa pratica può essere visto ripensando ai progetti di cui potresti aver sentito parlare (o di cui potresti aver fatto parte) in cui il processo di compilazione era settimanale o meno frequente e di solito portava a un "inferno di integrazione", in cui tutto si rompeva e nessuno sapeva perché. L'integrazione poco frequente porta a seri problemi in un progetto software. Innanzitutto, sebbene l'integrazione sia fondamentale per la distribuzione di un buon codice funzionante, il team non è esperto in materia e spesso viene delegata a persone che non hanno familiarità con l'intero sistema. In secondo luogo, il codice raramente integrato è spesso, direi di solito, codice buggato. I problemi si insinuano al momento dell'integrazione e non vengono rilevati da nessuno dei test che vengono eseguiti su un sistema non integrato. In terzo luogo, un processo di integrazione debole porta a lunghi blocchi del codice. I blocchi del codice implicano lunghi periodi di tempo in cui i programmatori potrebbero lavorare su importanti funzionalità distribuibili, ma che tali funzionalità devono essere trattenute. Ciò indebolisce la tua posizione sul mercato o con i tuoi utenti finali.

10. Proprietà collettiva del codice

In un progetto di Extreme Programming, qualsiasi coppia di programmatori può migliorare qualsiasi codice in qualsiasi momento. Ciò significa che tutto il codice beneficia dell'attenzione di molte persone, il che aumenta la qualità del codice e riduce i difetti. C'è anche un altro importante vantaggio: quando il codice è di proprietà di singoli individui, le funzionalità richieste vengono spesso inserite nel posto sbagliato, poiché un programmatore scopre di aver bisogno di una funzionalità da qualche parte nel codice che non possiede. Il proprietario è troppo impegnato per farlo, quindi il programmatore inserisce la funzionalità nel suo codice, dove non dovrebbe. Ciò porta a un codice brutto, difficile da mantenere, pieno di duplicazioni e con scarsa (cattiva) coesione. La proprietà collettiva potrebbe essere un problema se le persone lavorassero ciecamente su codice che non capiscono. XP evita questi problemi attraverso due tecniche chiave: i test del programmatore catturano gli errori e la programmazione in coppia significa che il modo migliore per lavorare su codice non familiare

è fare coppia con l'esperto. Oltre a garantire buone modifiche quando necessario, questa pratica diffonde la conoscenza in tutto il team.

11. Standard di codifica

I team XP seguono uno standard di codifica comune, in modo che tutto il codice nel sistema sembri scritto da un singolo individuo, molto competente. Le specifiche dello standard non sono importanti: ciò che conta è che tutto il codice sembri familiare, a supporto della proprietà collettiva.

12. Metafora

I team di Extreme Programming sviluppano una visione comune di come funziona il programma, che chiamiamo "metafora". Nella migliore delle ipotesi, la metafora è una semplice descrizione evocativa di come funziona il programma, come "questo programma funziona come un alveare di api, che esce per prendere il polline e lo riporta all'alveare" come descrizione di un sistema di recupero delle informazioni basato su agenti. A volte non nasce una metafora sufficientemente poetica. In ogni caso, con o senza immagini vivide, i team XP usano un sistema comune di nomi per essere sicuri che tutti capiscano come funziona il sistema e dove cercare per trovare la funzionalità che stai cercando, o per trovare il posto giusto dove mettere la funzionalità che stai per aggiungere.

13. Ritmo sostenibile

I team di Extreme Programming sono lì per il lungo termine. Lavorano sodo e a un ritmo che può essere sostenuto indefinitamente. Ciò significa che lavorano straordinari quando è efficace e che normalmente lavorano in modo da massimizzare la produttività settimana dopo settimana. Oggigiorno è abbastanza noto che i progetti di marcia della morte non sono né produttivi né producono software di qualità. I team XP sono lì per vincere, non per morire.

In XP, il design è mantenuto il più semplice possibile. Dal momento che il futuro è, dopo tutto, poco chiaro, non serve a niente progettare un grande schema che non sarà comunque seguito. Quindi il design copre solo la versione corrente del sistema. Dopo che un'attività è stata completata, il sistema viene controllato per vedere come può essere migliorato (rimuovendo il codice duplicato, rendendolo più semplice, rendendolo più flessibile). Questo si chiama *refactoring.* Il refactoring non deve essere limitato al proprio codice. Ognuno è responsabile dell'intero sistema. Per fare in modo che funzioni, il team deve definire standard di codifica.

Quando un team lavora all'implementazione di una storia, allo stesso tempo scrive dei test per verificare l'implementazione di quella storia. Prima che il nuovo codice venga archiviato, tutti questi test devono essere eseguiti correttamente. Dopo che il codice è stato archiviato, viene eseguita la suite di test completa e di nuovo tutti i test devono essere eseguiti

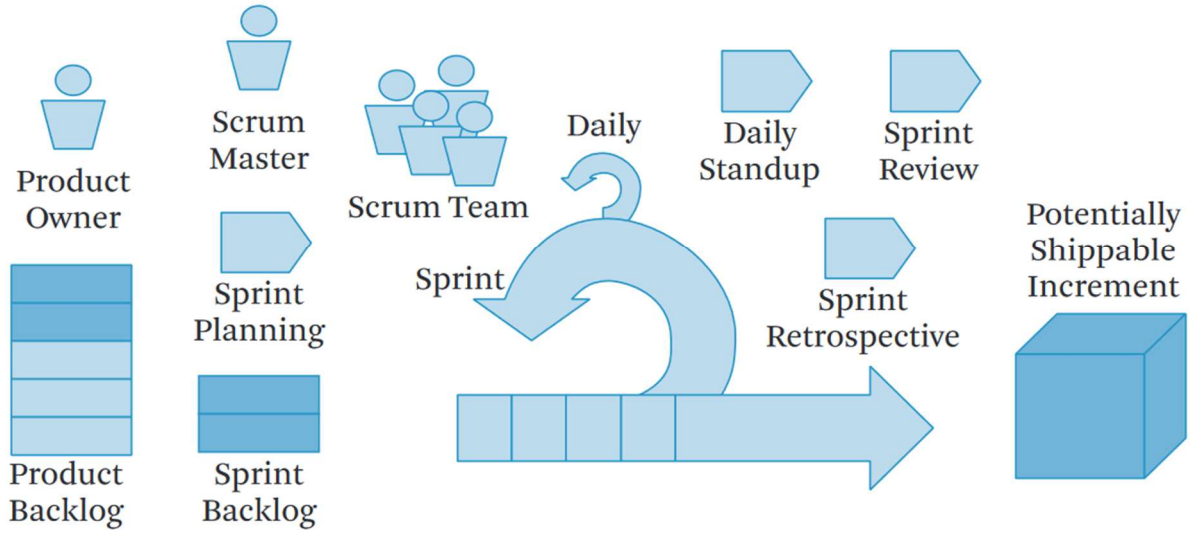
correttamente. In caso contrario, il nuovo codice viene rimosso per risolverlo. In questo modo, c'è sempre un sistema in esecuzione.

XP si basa su cinque principi che guidano le sue pratiche:

- **Feedback rapido Il feedback** si ottiene rapidamente, in poche ore o al massimo in pochi giorni. Man mano che ogni piccolo pezzo aggiunto viene testato, gli sviluppatori imparano immediatamente cosa funziona e cosa no. Poiché un sistema in esecuzione viene consegnato frequentemente al cliente, il cliente apprende quale valore offre il sistema e quali funzionalità sono necessarie successivamente.
- **Semplicità** Il lavoro di oggi è fatto oggi e il lavoro di domani è lasciato per domani. Non creare ulteriore complessità in modo che una determinata classe diventi più flessibile e possa essere riutilizzata se viene aggiunta una determinata funzionalità. Se e quando questa funzionalità sarà necessaria, verrà aggiunta e il codice verrà rifattorizzato per renderlo più semplice.
- **Cambiamento incrementale** In XP, le cose cambiano con piccoli incrementi. Il piano cambia un po' alla volta, il design cambia un po' alla volta, la squadra cambia un po' alla volta, ecc.
- **Abbracciare il cambiamento** Non pianificando, progettando o codificando più di quanto è necessario in questo momento, la maggior parte delle opzioni per il futuro vengono mantenute. Solo il problema più urgente viene affrontato oggi. Il resto è lasciato a domani.
- **Lavoro di qualità** La qualità è un must. Il team dovrebbe essere orgoglioso di offrire una qualità eccellente.

Come notato prima in questo capitolo, i metodi agili sono adatti per determinati progetti, ma non per tutti. Questo vale sicuramente anche per XP, l'approccio agile più estremo. Se i requisiti non sono sicuri, il sistema non è troppo grande e il cliente può essere sul posto, XP merita una seria considerazione. Le prime fonti consigliano di utilizzare tutte le pratiche di XP, poiché si rafforzano a vicenda. Ma al giorno d'oggi esistono anche molti approcci che adottano una o alcune delle pratiche di XP.

3.3 SCRUM



Scrum è un framework di sviluppo agile che aiuta le squadre a gestire e sviluppare prodotti complessi. Utilizza un approccio iterativo e incrementale per consegnare valore in modo più rapido ed efficace. I principali metodi di sviluppo Scrum includono:

1. **Product Backlog:** Il Product Owner definisce e priorizza i requisiti del progetto all'interno di un documento chiamato "Product Backlog". Questo rappresenta la lista delle funzionalità, dei bug e degli altri elementi necessari per il prodotto.
2. **Sprint Planning:** All'inizio di ogni sprint, la squadra e il Product Owner collaborano per selezionare gli elementi più importanti dal Product Backlog e stabiliscono gli obiettivi per lo sprint.
3. **Sprint:** Uno sprint è un periodo di tempo fisso, generalmente da 2 a 4 settimane, durante il quale la squadra lavora per consegnare gli elementi selezionati nel corso della pianificazione dello sprint.
4. **Daily Scrum:** Ogni giorno durante uno sprint, la squadra partecipa a una breve riunione chiamata "Daily Scrum" o "Stand-up meeting". Durante questa riunione, i membri della squadra condividono informazioni sul progresso, identificano eventuali ostacoli e pianificano le attività per il giorno successivo.
5. **Sprint Review:** Alla fine di ogni sprint, la squadra tiene una Sprint Review per presentare il lavoro completato al Product Owner e agli stakeholder. Questa è un'opportunità per ottenere feedback e apportare modifiche al Product Backlog.
6. **Sprint Retrospective:** Dopo la Sprint Review, la squadra partecipa a una Sprint Retrospective per riflettere sul processo e identificare miglioramenti che possono essere apportati nelle future iterazioni.
7. **Roles:** Scrum definisce ruoli chiave all'interno del team, tra cui il Product Owner, lo Scrum Master e i membri del team di sviluppo. Ognuno di questi ha responsabilità specifiche all'interno del framework. (vedremo meglio nell'organizzazione dei gruppi)

8. **Artefatti:** Gli artefatti principali in Scrum includono il Product Backlog, lo Sprint Backlog e il Product Increment. Questi documenti aiutano a gestire il lavoro e a garantire la trasparenza nel processo.

Scrum è progettato per promuovere la trasparenza, l'ispezione e l'adattamento continuo, consentendo alle squadre di rispondere in modo flessibile ai cambiamenti nei requisiti del progetto o nelle esigenze degli stakeholder. È ampiamente utilizzato nell'ambito dello sviluppo software ma può essere adattato per essere utilizzato in altri contesti in cui è necessaria la gestione di progetti complessi.

Uno strumento che si usa spesso in scrum è la **Kanban Board**.



Una Kanban board è una rappresentazione visiva di un sistema Kanban, un approccio alla gestione del lavoro e alla pianificazione delle attività che si basa su principi di flusso continuo e limitazione del lavoro in corso (WIP, Work in Progress). Questo sistema è stato originariamente sviluppato da Toyota per ottimizzare la produzione nelle fabbriche, ma è stato adattato con successo per l'uso in una vasta gamma di settori, inclusi lo sviluppo software, la gestione di progetti e altre attività lavorative.

La Kanban board è solitamente una lavagna o un quadro diviso in colonne e righe, con cartellini o note adesive che rappresentano le attività o le cose da fare. Le colonne sulla board rappresentano lo stato del lavoro, come "Da fare", "In corso" e "Completato". Le note o i cartellini vengono spostati da una colonna all'altra per riflettere lo stato di avanzamento delle attività.

Le principali caratteristiche di una Kanban board includono:

- Visualizzazione chiara: Una Kanban board fornisce una vista visiva del flusso di lavoro in modo che tutti i membri del team possano vedere quali attività sono in corso, quali sono completate e quali devono essere fatte successivamente.
- Limitazione del lavoro in corso (WIP): Le colonne sulla board spesso hanno limiti di quantità di lavoro che possono essere in corso contemporaneamente. Questo aiuta a evitare sovraccarichi di lavoro e a mantenere un flusso costante.
- Tracciamento del progresso: Gli spostamenti delle note o dei cartellini attraverso le colonne consentono al team di monitorare il progresso delle attività in modo efficiente.
- Rilevamento dei problemi: La Kanban board facilita la rilevazione di blocchi, ritardi o altri problemi nel flusso di lavoro, poiché le attività bloccate possono essere facilmente individuate.
- Aggiornamenti in tempo reale: La board viene aggiornata in tempo reale man mano che le attività vengono completate o avanzano, consentendo una comunicazione chiara tra i membri del team.

In sintesi, una Kanban board è uno strumento visuale che aiuta i team a gestire il flusso di lavoro, migliorare la produttività e la trasparenza, e identificare e risolvere eventuali problemi nel processo di lavoro.

3.4 IL PROCESSO RATIONAL UNIFIED PROCESS (RUP)

Il Rational Unified Process è un processo di sviluppo iterativo, orientato allo sviluppo di sistemi orientati agli oggetti. Viene fornito con un sacco di supporto per gli strumenti; ci sono fonti e modelli che un'azienda può caricare su una intranet; e ci sono fonti e modelli per vari tipi di documenti su Internet. Completa il linguaggio di modellazione unificato (UML). RUP potrebbe essere visto come un mezzo tra i metodi basati su documenti e agili. Ha un processo ben definito e include attività iniziali di ingegneria dei requisiti ragionevolmente estese, ma sottolinea il coinvolgimento delle parti interessate attraverso la sua natura guidata dai casi d'uso. La sua struttura di processo bidimensionale è illustrata nella Figura 3.6.

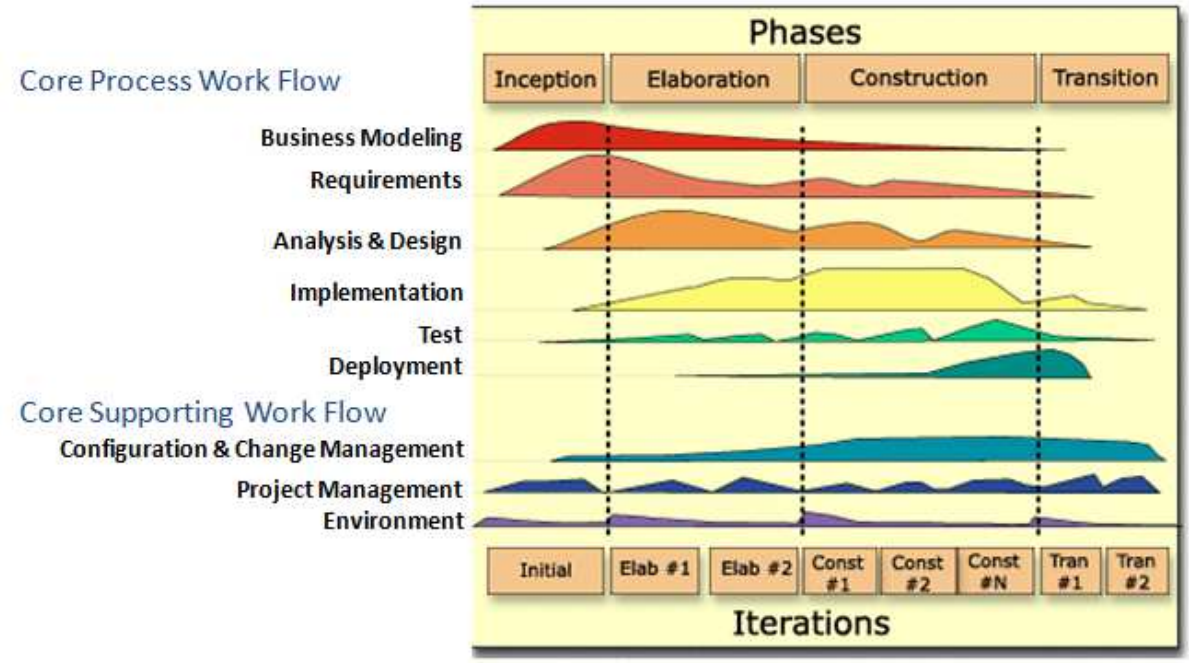


Figure 3.6 RUP Phase model

RUP distingue quattro fasi: inizio, elaborazione, costruzione e transizione. All'interno di ciascuna fase possono verificarsi diverse iterazioni. Lungo una seconda dimensione, RUP distingue nove "flussi di lavoro", come un flusso di lavoro dei requisiti e un flusso di lavoro di test. Questi flussi di lavoro raggruppano attività logiche e potrebbero estendersi a tutte le fasi, con livelli di attenzione diversi. Ad esempio, è probabile che il flusso di lavoro dei requisiti riceva molta attenzione durante le prime fasi, mentre il flusso di lavoro di distribuzione è più rilevante nella fase di transizione. Ciò è illustrato dalle forme ondulate accanto a ciascun flusso di lavoro nella Figura 3.6, che ci consente di distinguere tra iterazioni successive e sottolineare che iterazioni diverse hanno un'enfasi diversa. Riconosce che l'ingegneria dei requisiti, la progettazione e altri flussi di lavoro sono attività in corso piuttosto che fasi con un orario di inizio e fine rigoroso.

La fase iniziale si concentra sull'ottenimento di obiettivi chiari: qual è lo scopo di questo progetto, quali sono i suoi confini e quali sono i criteri di accettazione che verranno utilizzati quando il sistema sarà consegnato ai propri clienti? Anche in questa fase vengono stimati i costi complessivi, le tempistiche ei rischi. Vengono sviluppati casi d'uso critici e un'architettura candidata. Al termine di questa fase, il business case per il sistema deve essere chiaro. Questo potrebbe essere l'input per una decisione go/no-go (una decisione se andare avanti o meno con lo sviluppo).

La fase di elaborazione è principalmente finalizzata all'analisi del dominio del problema e all'ottenimento di una solida architettura. Al termine di questa fase, verrà identificata la maggior parte dei casi d'uso e tutti i principali rischi dovranno essere risolti.

La fase di costruzione è un processo di fabbricazione, costruzione. L'enfasi è sullo sviluppo di prodotti implementabili. I componenti completi sono sviluppati e accuratamente testati. I manuali dell'utente sono scritti. Al termine di questa fase è pronta la prima versione operativa del sistema, la *versione beta.*

Nella fase di transizione, il sistema viene rilasciato alla comunità degli utenti e sottoposto a beta test. Durante questa fase, potrebbe essere necessario convertire i database, formare gli utenti e, in caso di sostituzione di un sistema, il sistema legacy da sostituire viene gradualmente eliminato.

1. **Ideazione** (Inception)	Definizione dell'ambito e degli obiettivi del progetto. Comprensione iniziale dei requisiti e valutazione della fattibilità
2. **Elaborazione** (Elaboration)	Specifica dettagliata delle caratteristiche del sistema e sviluppo della baseline architetturale. Creazione di un prototipo e di un'architettura robusta
3. **Costruzione** (Construction)	Sviluppo del software, iterazione e realizzazione delle funzionalità. Implementazione e integrazione del codice per completare il prodotto
4. **Transizione** (Transition)	Rilascio del prodotto all'utente finale. Gestione del passaggio, che include beta-testing, training e supporto

Tabella 3.5 Pratiche di RUP

Pratica	Descrizione
Sviluppo iterativo	I sistemi sono sviluppati in modo iterativo. Questo non è un processo incontrollato. Le iterazioni sono pianificate e il progresso viene misurato attentamente.
Gestione dei requisiti	RUP ha un approccio sistematico per sollecitare, acquisire e gestire i requisiti, comprese le possibili modifiche a questi requisiti.
Architettura e uso dei componenti	Le prime fasi di RUP si traducono in un'architettura. Questa architettura viene utilizzata nel resto del progetto. È descritto in diversi punti di vista. RUP supporta lo sviluppo di sistemi basati su componenti, in cui ogni componente è un software non banale con confini ben definiti.
Modellazione e UML	Gran parte del RUP riguarda lo sviluppo di modelli, come un modello di caso d'uso, un modello di test, ecc. Questi modelli sono descritti in UML.

Pratica	Descrizione
Qualità del processo e del prodotto	La qualità non è un componente aggiuntivo, ma è responsabilità di tutti i soggetti coinvolti. Il flusso di lavoro dei test ha lo scopo di verificare che il livello di qualità atteso sia soddisfatto.
Configurazione e gestione delle modifiche	I progetti di sviluppo iterativi forniscono una grande quantità di prodotti, molti dei quali vengono modificati di frequente. Ciò richiede procedure valide per farlo e un supporto adeguato per gli strumenti.
Sviluppo basato sui casi d'uso	I casi d'uso descrivono il comportamento del sistema. Svolgono un ruolo importante in vari flussi di lavoro, in particolare i requisiti, la progettazione, i test e il flusso di lavoro di gestione.
Configurazione del processo	Una taglia non va bene per tutti. Sebbene RUP possa essere utilizzato "così com'è", può anche essere modificato e adattato per adattarsi meglio a circostanze specifiche.
Supporto per gli strumenti	Per essere efficace, una metodologia di sviluppo software necessita del supporto degli strumenti. RUP è supportato da un'ampia varietà di strumenti, in particolare nell'area della modellazione visiva e della gestione della configurazione.

RUP si basa su una serie di migliori pratiche (vedi **Tabella 3.5**) che si sono evolute nel corso degli anni. Molte di queste best practices, ovviamente, sono presenti anche in altri modelli di sviluppo. Un punto di forza di RUP è che fornisce un'integrazione equilibrata delle migliori pratiche. Dato il suo background, non sorprende che RUP sia orientato allo sviluppo di sistemi orientati agli oggetti. Ma RUP è adatto per progetti con caratteristiche molto diverse. L'adattamento di RUP alla situazione in questione, tuttavia, è lasciato all'utente del metodo.

3.5 MDA: MODEL DRIVEN ARCHITECTURE

Negli approcci tradizionali allo sviluppo del software, costruiamo tutti i tipi di modelli durante l'ingegneria dei requisiti e la progettazione. Questi modelli possono essere espressi più o meno formalmente. Di solito, i modelli non sono così formali. Esempi tipici includono un elenco di requisiti in linguaggio naturale, un diagramma box-and-line che illustra un progetto globale e diagrammi UML; vedi anche **il capitolo 10**. Il modello definitivo è il codice sorgente, che è il più formale possibile.

Le transizioni tra i vari modelli vengono eseguite manualmente. Cioè, traduciamo manualmente un modello dei requisiti in un modello di progettazione. Il modello di

progettazione viene successivamente tradotto manualmente in codice. Qualsiasi evoluzione viene compiuta modificando l'ultimo modello della catena, ovvero il codice sorgente. Poiché di solito abbiamo poco tempo, i requisiti e i modelli di progettazione precedenti spesso non vengono aggiornati, in modo che diventino rapidamente obsoleti.

Nell'architettura basata su modelli (MDA) [1], l'attenzione si sposta dalla scrittura di codice allo sviluppo di modelli. Il modello progettuale viene catturato in un formalismo molto preciso e tradotto *automaticamente* in codice sorgente. L'evoluzione successiva avviene quindi a livello di modellizzazione. Una volta apportate le modifiche, viene generata automaticamente la versione successiva del sistema. Quindi la manutenzione non viene eseguita a livello di codice sorgente, ma a livello di modello. Questo cambio di paradigma è illustrato nella **Figura 3.7** .

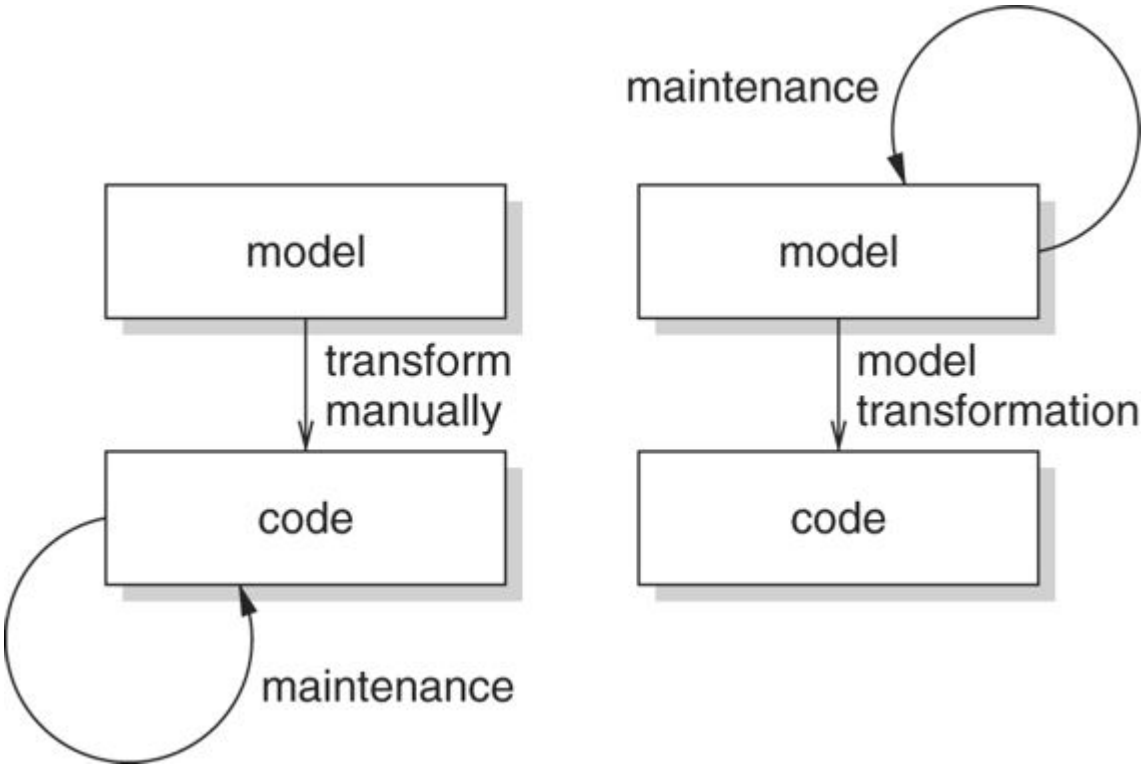


Figure 3.7 Manutenzione basata su codice e manutenzione basata su modello

In MDA si distinguono diversi modelli. I principali sono il modello indipendente dal calcolo (CIM), il modello indipendente dalla piattaforma (PIM) e il modello specifico della piattaforma (PSM). Il CIM è il modello più astratto; cattura i requisiti. Il CIM viene trasformato nel PIM, che descrive la funzionalità di un sistema, senza alcun dettaglio specifico della piattaforma. Il PSM include i dettagli delle piattaforme di destinazione, come.NET o J2EE. Infine, il PSM viene trasformato in codice eseguibile.

Le trasformazioni da PIM a PSM e da PSM a codice sono automatizzate. La trasformazione da CIM a PIM non è automatizzata. Molte discussioni su MDA si concentrano sulle parti completamente automatizzate e tralasciano del tutto il CIM. Le trasformazioni del modello da

PIM a PSM e da PSM a codice sono definite in un linguaggio specifico che definisce come i concetti di un livello vengono mappati sui concetti di un altro livello. Per ogni combinazione di PIM e PSM, deve essere definita una tale trasformazione del modello. E allo stesso modo per ogni combinazione di PSM da codificare. Tuttavia, una volta definita tale mappatura, può essere riutilizzata più e più volte.

MDA è un'iniziativa dell'Object Management Group (OMG), un'organizzazione no-profit responsabile anche dell'UML (vedi **Capitolo 10**). La visione di MDA di sviluppare software a livello di modelli astratti e automatizzare l'effettiva generazione del codice non è nuova. I tentativi precedenti includono specifiche eseguibili, linguaggi di quarta generazione e così via. Questi primi tentativi sono in gran parte falliti. Alcuni critici affermano che nemmeno l'MDA funzionerà. Ma il discorso è ancora aperto.

3.6 INTERMEZZO: MANUTENZIONE O EVOLUZIONE

- I vecchi programmi sui salari non muoiono mai; mettono su pancia.
  Robert Granholm (Datamation, 1971)

Nel **Capitolo 1** è stato sottolineato che un notevole sforzo di manutenzione è inevitabile. Ogni attività di manutenzione, che riguardi la riparazione di un errore o l'adattamento di un sistema alle nuove esigenze dell'utente, comporta in linea di principio tutti gli aspetti del ciclo di sviluppo iniziale. Durante la manutenzione, dobbiamo anche analizzare il problema e concepire un progetto che viene successivamente implementato e testato.

La prima grande differenza è che queste modifiche vengono apportate a un prodotto esistente. Tuttavia, durante lo sviluppo iniziale spesso non partiamo nemmeno da zero. Se un'organizzazione esistente decide di automatizzare la gestione degli ordini, il sistema potrebbe dover interfacciarsi con sistemi già esistenti, ad esempio, per l'amministrazione delle scorte e la contabilità. Pertanto, le attività di manutenzione differiscono in gradi dallo sviluppo iniziale, piuttosto che fondamentalmente. Questa differenza relativa è ancora più evidente quando il sistema viene prototipato o sviluppato in modo incrementale.

La seconda differenza principale, la pressione del tempo, ha un impatto molto maggiore. La pressione del tempo è più sentita quando si riparano gli errori, poiché in tal caso è del tutto possibile che alcune parti dell'organizzazione debbano chiudere perché il software non è operativo. In questi casi, dobbiamo lavorare contro il tempo per identificare e riparare gli errori; spesso saltiamo un'analisi approfondita e una fase di progettazione e semplicemente aggiorniamo il codice. La struttura del sistema tende a soffrire di tali patch. L'entropia del sistema aumenta, il che ostacola le successive attività di manutenzione. Peggio ancora, la documentazione del sistema potrebbe non essere aggiornata. Il software e la documentazione corrispondente si allontanano, il che ostacola anche le future attività di manutenzione. Una discussione più elaborata sui problemi di manutenzione è fornita nel **Capitolo 14**.

Lehman ei suoi collaboratori hanno studiato a fondo le dinamiche dei sistemi software che devono essere mantenuti e crescere di dimensioni. Sulla base di questi studi quantitativi, hanno formulato le seguenti leggi dell'evoluzione del software:

1. **Cambiamenti continui** Un sistema in uso subisce continue modifiche, fino a quando non viene giudicato più conveniente ristrutturare il sistema o sostituirlo con una versione completamente nuova.
2. **Complessità crescente** Un programma che viene modificato diventa sempre meno strutturato (l'entropia aumenta) e quindi diventa più complesso. Bisogna investire uno sforzo extra per evitare una crescente complessità. (vedi concetto di technical dept)
3. **Autoregolazione** I processi di evoluzione del software si autoregolano e promuovono una crescita regolare del software.
4. **Conservazione della stabilità organizzativa (tasso di lavoro invariante)** L'andamento complessivo dei progetti di sviluppo software è statisticamente invariante.
5. **Conservazione della familiarità** Un sistema sviluppa un incremento di crescita costante per sostenere la familiarità dell'organizzazione con il sistema. Quando questo incremento viene superato, si verificheranno problemi di qualità e utilizzo.
6. **Crescita continua** La funzionalità di un sistema deve aumentare continuamente per mantenere la soddisfazione dell'utente.
7. **Qualità in declino** La qualità di un sistema diminuisce a meno che non venga attivamente mantenuta e adattata al suo ambiente in evoluzione.
8. **Sistema di feedback** L'evoluzione del software deve essere vista come un sistema di feedback per ottenere miglioramenti.

In una delle prime pubblicazioni, Lehman (**1974**) confronta la crescita dei sistemi software con quella delle città e delle burocrazie. Fa una distinzione tra attività progressiste e antiregressive nello sviluppo del software. Lehman considera questo modello applicabile anche ai sistemi socio-economici. In una città, ad esempio, le attività progressiste contribuiscono ad aumentare il tenore di vita o la qualità della vita. Le attività antiregressive, come la raccolta dei rifiuti, servono a mantenere lo status quo. Se si presta un'attenzione insufficiente a queste attività antiregressive, si verificherà un declino. Le attività antiregressive spesso non sono interessanti, politicamente parlando. È un investimento nel futuro, che è meglio lasciare ad altri. (Lo stesso fenomeno può essere osservato nella crescita dell'industria chimica e nei conseguenti problemi di inquinamento.)

Secondo Lehman, gli stessi tipi di attività si verificano all'interno di un progetto di sviluppo software. La generazione di nuovo codice e la modifica del codice esistente sono attività progressive. Queste sono attività interessanti, stimolanti e gratificanti. Forniscono all'utente funzionalità nuove o migliori. Scrivere documentazione, migliorare la struttura del codice e mantenere una buona comunicazione tra le persone coinvolte sono attività antiregressive. Trascurare queste attività potrebbe non essere dannoso a breve termine, ma sicuramente lo sarà a lungo termine. Per ogni sistema, dobbiamo cercare un giusto equilibrio tra questi tipi di attività.

La legge di autoregolazione è illustrata dalla **Figura 3.8** che illustra il modello di crescita degli attributi del sistema nel tempo. Gli attributi del sistema includono la lunghezza (misurata in righe di codice), il numero di moduli, il numero di funzioni richiamabili dall'utente, ecc. L'asse del tempo può indicare il numero di rilascio, il numero di mesi di funzionamento del sistema o una misura simile. (I dati effettivi studiati da Lehman riguardavano la relazione tra il numero di moduli e il numero di versione del sistema operativo OS360.)

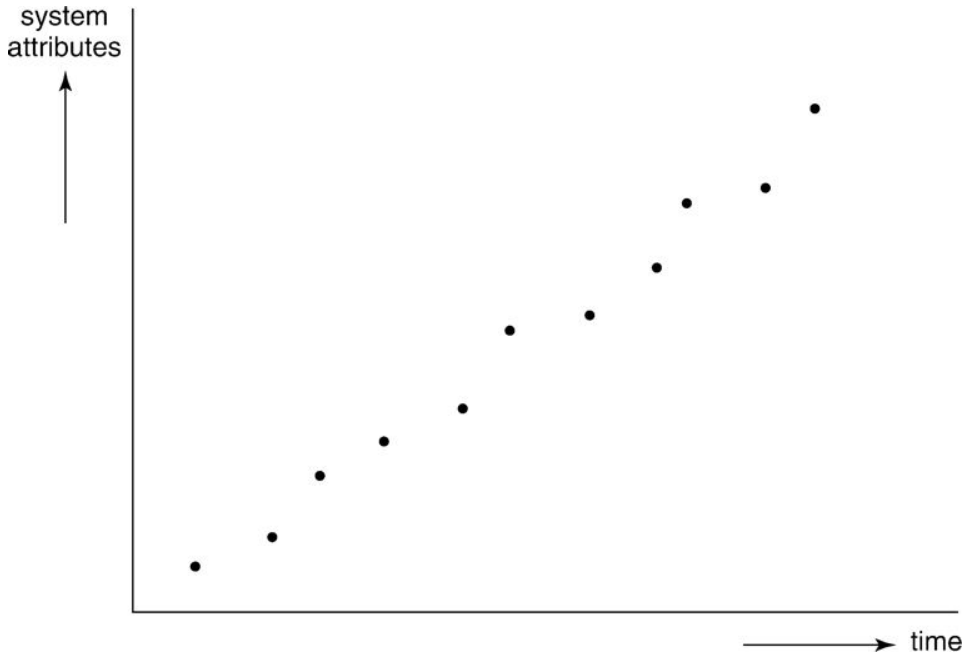


Figura 3.8 Crescita degli attributi di sistema nel tempo

La relazione illustrata nella **Figura 3.8** è quasi lineare. Anche le increspature nella figura sono molto regolari. Periodi di crescita più che lineare si alternano a periodi di crescita meno che lineare. Lehman spiega i periodi di crescita più che lineare indicando la pressione degli utenti per ottenere più funzionalità il più rapidamente possibile.

Gli sviluppatori oi manutentori tendono a piegarsi sotto questa pressione. Di conseguenza, usano trucchi e scorciatoie nel codice, la documentazione è in ritardo, vengono introdotti errori e il sistema non è sufficientemente testato. Dopo un po', viene prestata maggiore attenzione alle attività antiregressive: il codice deve essere rifattorizzato e la documentazione aggiornata prima che sia possibile un'ulteriore crescita. I due tipi di attività si stabilizzano nel tempo.

La legge di conservazione della stabilità organizzativa sembra a prima vista piuttosto sorprendente. Lehman e Belady (**1985**) hanno scoperto che cose come la manodopera e altre risorse non sono affatto correlate alla velocità con cui i sistemi crescono o cambiano. Apparentemente, i grandi sistemi sono in una sorta di stato saturo. Più persone possono essere mantenute al lavoro ma, a lungo termine, non hanno alcun impatto percepito sull'evoluzione del sistema.

Una crescita superiore alla media in alcune versioni di un sistema era, nelle osservazioni di Lehman e Belady, quasi sempre seguita da una crescita inferiore alla media nella versione successiva (come espresso nella legge di conservazione della familiarità). In uno dei sistemi che hanno studiato, una crescita sostanzialmente maggiore ha portato inevitabilmente a problemi: minore affidabilità, costi più elevati, ecc. Apparentemente, un'organizzazione deve mantenere una sufficiente dimestichezza con il proprio software. Anche in questo caso è stato osservato un feedback autoregolante.

Dalla discussione precedente, ne consegue che dobbiamo essere vigili durante la manutenzione. Dobbiamo preservare la qualità in ogni fase. Possiamo cercare di precludere i pericoli sopra delineati impegnandoci esplicitamente nelle varie fasi di sviluppo durante la manutenzione. Il processo ciclico seguito durante lo sviluppo iniziale si verifica poi anche durante la manutenzione. Come per la prototipazione, il tempo necessario per completare il ciclo sarà, in generale, molto più breve rispetto allo sviluppo iniziale. Questo modo di considerare la manutenzione ricorda da vicino la visione evolutiva dello sviluppo del software. La realizzazione della prima versione di un sistema software è solo il primo passo. Abbastanza vero, questo primo passaggio è più costoso della maggior parte dei passaggi che seguono, ma non è fondamentalmente diverso. Nel **Capitolo 1** abbiamo già notato che mettere la manutenzione alla pari con lo sviluppo può anche avere effetti positivi sull'ambiente sociale e organizzativo in cui avviene lo sviluppo del software.

Il modello a cascata ci offre una vista *statica* del sistema da sviluppare. La realtà è diversa. Nello sviluppo del software, e in particolare durante la manutenzione, ci occupiamo di un sistema in evoluzione. Come abbiamo notato prima: il software non si costruisce, cresce.

3.7 SOFTWARE PRODUCT LINES - LINEE DI PRODOTTI SOFTWARE

Quando vengono sviluppati prodotti simili, potremmo sperare di **riutilizzare** elementi di prodotti precedenti durante lo sviluppo di nuovi prodotti. Tuttavia, questa non è l'abitudine nello sviluppo di software. In molte organizzazioni non vi è alcun incentivo a riutilizzare elementi (codice, design o qualsiasi altro artefatto) di un altro sistema poiché non è ciò per cui veniamo pagati. Allo stesso modo, non vi è alcun incentivo a produrre elementi riutilizzabili, poiché il presente progetto è tutto ciò che conta.

In alternativa, possiamo concepire la nozione di una *linea di prodotti software,* un insieme di sistemi software che condividono elementi. In una linea di prodotti software, il riutilizzo è pianificato, non accidentale. Per mantenere l'ambito entro limiti ragionevoli, il riutilizzo pianificato è legato a un determinato dominio.

Le linee di prodotti software (Software Product Line), o sviluppo di linee di prodotti software, si riferiscono a metodi, strumenti e tecniche di ingegneria del software per creare una raccolta di sistemi software simili da un set condiviso di risorse software utilizzando un mezzo di produzione comune (ad esempi un unico codice sorgente).

Il Carnegie Mellon Software Engineering Institute definisce una linea di prodotti software come "un insieme di sistemi software-intensive che condividono un insieme comune e gestito di funzionalità che soddisfano le esigenze specifiche di un particolare segmento di mercato o missione e che sono sviluppati da un insieme comune di risorse di base in un modo prescritto".

Supponiamo di aver sviluppato un sistema bibliotecario di successo per la nostra biblioteca della facoltà di informatica. È probabile che ci verrà chiesto di sviluppare un sistema simile per, diciamo, la facoltà di scienze della terra. Riutilizziamo quanto più possibile dal nostro primo sistema, ma è probabile che siano necessari alcuni perfezionamenti per soddisfare l'altra facoltà. Possono avere mappe che possono essere prese in prestito e che richiedono una gestione specifica. Un'altra facoltà può venire e chiedere un terzo sistema. E così via. Piuttosto che essere reattivi e riutilizzare elementi adeguati degli sforzi precedenti, potremmo anche essere proattivi e pianificare lo sviluppo di una serie di sistemi nel dominio dell'automazione delle biblioteche fin dall'inizio. Ciò comporta due processi: **ingegneria del dominio** e **ingegneria dell'applicazione**.

Nell'ingegneria del dominio, analizziamo il dominio per il quale svilupperemo. Questo processo ha un proprio ciclo di vita. Ne risulta un insieme di componenti riutilizzabili che costituiscono la base per i prodotti da sviluppare. Solitamente viene prodotta anche un'architettura di riferimento per tutti i prodotti da sviluppare. Un passo importante in questo processo è decidere l'ambito della linea di prodotti. Svilupperemo una linea di prodotti solo per le biblioteche universitarie o per le biblioteche in generale? Il primo è più semplice ma ha un mercato più limitato. Quest'ultimo ha potenzialmente un mercato più ampio, ma è probabile che si traduca in un'architettura complessiva più complessa e prodotti più complessi. L'ambito delle linee di prodotti è un problema difficile. È influenzato dalla strategia dell'organizzazione e richiede informazioni sulla probabile evoluzione del dominio. Infine, il processo di ingegneria del dominio produce un piano di produzione, una guida su come sviluppare prodotti all'interno della famiglia di prodotti.

L'ingegneria delle applicazioni riguarda lo sviluppo di singoli prodotti. Di solito segue uno schema simile a una cascata. I suoi input sono gli output del processo di ingegneria del dominio: l'architettura di riferimento, il piano di produzione e l'insieme delle risorse riutilizzabili.

Le organizzazioni della linea di prodotti spesso separano le attività di ingegneria del dominio dalle attività di ingegneria delle applicazioni. In effetti, queste attività costituiscono quindi progetti separati. Lo sviluppo di un singolo prodotto può comportare componenti nuovi o adattati che portano ad adattamenti a livello di famiglia di prodotti, che a loro volta influiscono sullo sviluppo dei prodotti successivi. Di conseguenza, ci sono cicli di feedback dal processo di ingegneria dell'applicazione al processo di ingegneria del dominio e viceversa.

Le linee di prodotti software sono particolarmente adatte in domini in cui vi sono molte variazioni in prodotti abbastanza simili, come telefoni cellulari, televisori o fotocamere. Le aziende che operano in questi domini hanno aperto la strada al campo della linea di prodotti.

Una discussione più elaborata delle linee di prodotti software e di altri approcci basati sui componenti che implicano processi di sviluppo separati per componenti e applicazioni è data nel **Capitolo 18**.

	Spazio del problema	Spazio della soluzione
Nell'ingegneria del **dominio**, analizziamo il dominio per il quale svilupperemo	Analisi del dominio	- Set di componenti riutilizzabili - Architettura di riferimento
L'ingegneria delle applicazioni riguarda lo sviluppo di singoli prodotti	Analisi dei requisiti	Derivazione del prodotto

3.8 MODELLAZIONE DI PROCESSO

- Senza un processo ripetibile, gli unici risultati ripetibili che potresti produrre sono errori. (Macala *et al.*, **1996**)

Negli anni '80, Osterweil ha lanciato l'idea di descrivere i processi di sviluppo del software come programmi. Questi **programmi di processo** sono scritti in un **linguaggio di programmazione di processo.** Come altri linguaggi di programmazione, i linguaggi di programmazione di processo hanno una sintassi e una semantica rigorosamente definite. Come semplice esempio, si consideri la descrizione in stile Pascal di un processo di revisione nella **Figura 3.9** .[2] Descrive le fasi consecutive di un processo di revisione. Il processo ha due input: il documento da rivedere e un numero che funge da soglia. La routine restituisce un valore booleano che indica se deve essere pianificata o meno un'altra revisione.

```
function review (document, threshold): Boolean;
begin prepare-review;
    hold-review(document, no-of-problems);
    make-report;
    return no-of-problems < threshold
end review;
```

Figure 3.9 Un programma di processo per il processo di revisione

Nella **Figura 3.9** , il processo di revisione è descritto in termini di successive attività da svolgere: prima viene preparata la revisione, poi si tiene la riunione e infine viene redatta una relazione. Possiamo anche descrivere il processo in termini di stati in cui può trovarsi. Dopo le attività di preparazione (distribuzione del documento tra i partecipanti, pianificazione di un

incontro, e così via), il documento è pronto per essere rivisto. Al termine della riunione, è possibile redigere una relazione. Dopo che il rapporto è stato scritto, è possibile eseguire ulteriori passaggi. La **Figura 3.10** usa la notazione UML per i diagrammi delle macchine a stati (vedi **Sezione 10.3.2**) per descrivere il processo di revisione in termini di stati e transizioni tra stati. La casella denominata processo di revisione descrive il processo di revisione vero e proprio. Gli ingressi e le uscite del processo sono indicati da frecce che conducono all'interno e all'esterno della scatola.

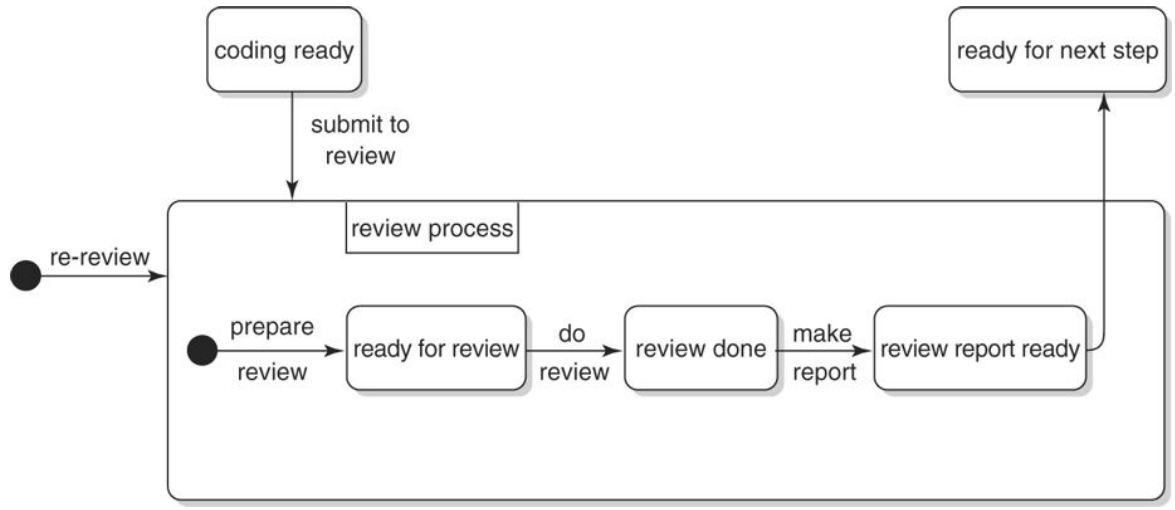


Figure 3.10 Diagramma di transizione dello stato del processo di revisione

Le reti di Petri forniscono ancora un altro formalismo per descrivere i modelli di processo. La **Figura 3.11** fornisce una vista della rete di Petri del processo di revisione. Una rete di Petri è un grafo diretto con due tipi di nodi: luoghi e transizioni. Un luogo è rappresentato come un cerchio. Denota uno stato (parziale) del sistema. Un luogo è contrassegnato o non contrassegnato. Nella **Figura 3.11** , il codice del luogo pronto è contrassegnato ma la revisione programmata non lo è. Una transizione è rappresentata da una linea retta. Una transizione riceve l'input da una o più posizioni e fornisce l'output a una o più posizioni. Gli ingressi e le uscite sono indicati da frecce che portano da e verso una transizione. Una transizione indica un'attività che può essere eseguita (nella terminologia di Petri net, 'licenziata') se tutte le sue posizioni di input sono contrassegnate. I luoghi possono quindi essere pensati come presupposti per le attività. Nella **Figura 3.11** , la riunione di revisione non può essere tenuta, poiché non è stata ancora programmata. Una volta programmato, il luogo corrispondente viene contrassegnato e la transizione può essere avviata. I contrassegni vengono quindi rimossi da tutte le posizioni di input e tutte le posizioni di output vengono invece contrassegnate.

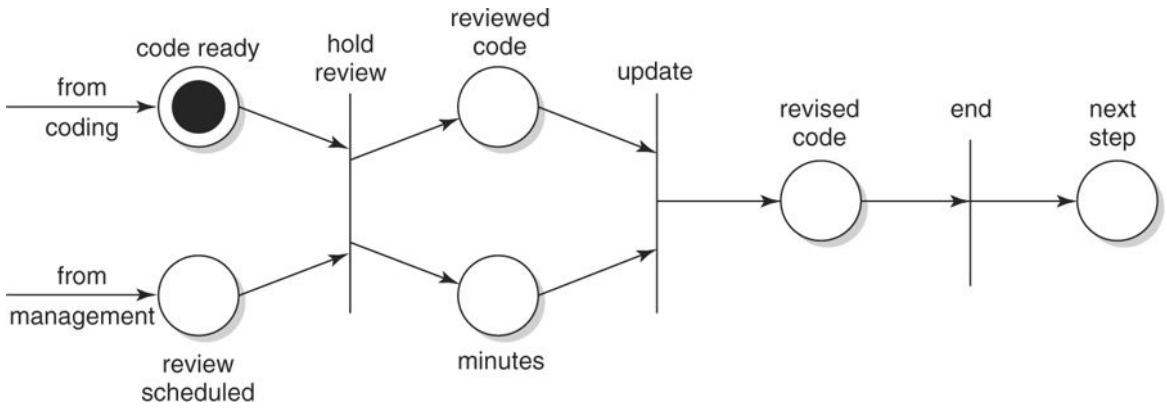


Figura 3.11Vista mediante reti di Petri del processo di revisione

Le reti di Petri sono una tecnica di modellazione interessante per i processi, poiché consentono una certa quantità di non determinismo e parallelismo. Ad esempio, il processo nella **Figura 3.11** non specifica l'ordine in cui devono essere eseguite le attività di codifica e programmazione. Possono andare avanti in parallelo; la sincronizzazione avviene al termine di entrambi.

Una descrizione precisa del processo software, sia in una notazione in un linguaggio di programmazione, in una notazione grafica o in altro, ha tre scopi principali:

- Facilita la comprensione e la comunicazione. In un progetto di sviluppo software, le persone devono lavorare insieme. Devono quindi avere una visione condivisa dei processi da svolgere e dei ruoli che devono svolgere in tali processi. Entrambi i modelli del processo di revisione sopra indicati possono essere utilizzati a questo scopo.
- Supporta la gestione e il miglioramento dei processi. Una descrizione precisa delle attività da svolgere può essere utilizzata dal project management per assegnare compiti e tenere traccia di chi sta facendo cosa. Se il processo di sviluppo del software deve essere migliorato, devi prima sapere qual è il processo attuale, cioè deve essere modellato.
- Può servire come base per il supporto automatizzato. Questo supporto automatizzato può guidare o far rispettare l'ordine in cui vengono eseguite le attività. Ad esempio, un revisore può ricevere automaticamente un messaggio che indica che un determinato pezzo di codice è pronto per la revisione non appena il suo autore rilascia quel codice. Il supporto automatizzato può anche essere utilizzato per monitorare e tenere traccia dei progressi, raccogliere informazioni e metriche di processo e così via.

La descrizione del processo di revisione nella **Figura 3.9** è molto deterministica. Può essere completamente automatizzato ed eseguito senza l'intervento umano. In generale, il lavoro che viene modellato sarà svolto sia dall'uomo che dalle macchine. Il termine neutro **enactment** è usato per denotare l'esecuzione di un processo da parte di esseri umani o macchine. Il supporto per l'attuazione dei processi è spesso combinato con il supporto per la gestione della configurazione (vedere **Sezione 4.1**).

Sebbene la modellazione precisa del processo software presenti vantaggi definiti, la formalità del processo risultante, o addirittura la rigidità, presenta anche alcuni pericoli e limitazioni:

- Molti aspetti del processo di sviluppo del software sono di natura euristica o creativa e non si prestano a una descrizione algoritmica. Ad esempio, i processi di debug o di progettazione effettivi saranno piuttosto difficili da acquisire in un modello di processo.
- Un modello di processo è un modello e, quindi, una semplificazione della realtà. Ad esempio, i suddetti modelli del processo di revisione non specificano cosa fare se il verbale della riunione non viene consegnato o la revisione non si tiene perché l'autore del codice è in congedo per malattia e così via.
- I modelli di processo spesso si concentrano sulla trasformazione di artefatti, come codice, una specifica dei requisiti o un piano di test. La progressione delle fasi attraverso le quali evolve l'artefatto può essere confusa con l'organizzazione dei processi attraverso i quali le persone sviluppano effettivamente quegli artefatti. Questo argomento è stato utilizzato in precedenza quando abbiamo criticato il modello a cascata. È supportato dagli studi di Zelkowitz (**1988**) e Guindon e Curtis (**1988**), riportati nella **Sezione 3.1** . Parnas e Clements (**1986**) usano argomentazioni simili quando criticano l'idea che il processo di progettazione del software sia razionale.
- I processi che non trasformano direttamente gli artefatti (ad esempio, l'apprendimento del dominio dell'applicazione, la gestione dei requisiti che fluttuano o sono in conflitto e la gestione delle interruzioni nella comunicazione o nel coordinamento) tendono a essere ignorati (Curtis *et al.*, **1988**).
- I processi sono trattati come discreti piuttosto che continui nel tempo (cioè ogni progetto invoca un processo separato). Questo punto di vista inibisce il trasferimento di conoscenze tra progetti, come discusso nella sezione precedente.

La modellazione dei processi ha ricevuto molta attenzione nella letteratura di ricerca. È indicativo della necessità di approcci più formali alla descrizione del processo software. L'ultima tendenza nella ricerca sulla modellazione dei processi mira a fornire agli sviluppatori guida e assistenza informatica, piuttosto che cercare di automatizzare completamente il processo. Descrizioni così precise forniscono una base per una serie di funzioni di supporto, che vanno dall'emanazione delle fasi di progettazione alla gestione dell'agenda. Questa tendenza a supportare le persone piuttosto che subentrare si adatta bene anche agli sviluppi agili.

3.9 RIEPILOGO

In questo capitolo abbiamo affrontato nuovamente il ciclo di vita del software. Ci sono parecchi argomenti contro il rigoroso ordinamento sequenziale delle fasi come discusso nel **Capitolo 1** . L'approccio tradizionale è, in larga misura, basato sui documenti. Lungo il percorso dall'inizio alla fine vengono individuate alcune pietre miliari. Il raggiungimento di tali traguardi è determinato dalla disponibilità di determinati documenti. Questi documenti svolgono quindi un ruolo chiave nel controllo del processo di sviluppo. È un processo pesante, guidato dalla pianificazione.

La pratica quotidiana difficilmente si adatta a questo modello. Il cambiamento è inevitabile e faremmo meglio ad adottare un metodo di sviluppo che si adatta al cambiamento. Negli ultimi anni sono stati proposti numerosi metodi agili e leggeri che affrontano consapevolmente il cambiamento. Questi si sono evoluti da metodi come la prototipazione, lo sviluppo incrementale e lo sviluppo rapido di applicazioni. Un metodo agile molto influente è eXtreme Programming, o XP.

Se una serie di prodotti simili viene sviluppata all'interno di un dominio, conviene pianificare il riutilizzo in anticipo, piuttosto che lasciare che siano i singoli progetti a fornire componenti riutilizzabili. Ciò ha portato alla nozione di linee di prodotti software, discussa nella **Sezione 3.6** . Nell'ingegneria della linea di prodotti software, l'ingegneria del dominio si occupa dello sviluppo di risorse riutilizzabili mentre l'ingegneria delle applicazioni produce singoli prodotti utilizzando tali risorse.

Infine, abbiamo introdotto la nozione di modellazione di processo, che ha lo scopo di descrivere il processo di sviluppo del software in modo preciso e non ambiguo. Tali descrizioni non intendono sostituire completamente le attività umane, ma piuttosto supportarle.

3.10 ULTERIORI LETTURE

Il modello a cascata è generalmente attribuito a Royce (**1970**) e divenne famoso grazie a Boehm (**1976**). Tuttavia, un approccio chiaramente graduale allo sviluppo del software, inclusa l'iterazione e il feedback, può essere trovato in pubblicazioni precedenti: (Benington, **1983**) e (Hosier, **1961**).

Vantaggi e svantaggi della prototipazione, sulla base di un'analisi di 22 casi di studio pubblicati e 17 resoconti di prima mano, sono riportati in (Gordon e Bieman, **1994**). (Verner e Cerpa, **1997**) affronta le diverse opinioni di analisti e manager sui pro e contro della prototipazione.

Per una discussione molto elaborata sulla RAD, vedere (Martin, **1991**). DSDM è discusso in (Stapleton, **2003**). Il design partecipativo è descritto in (Floyd *et al.*, **1989**). (CACM, **1993a**) è un numero speciale sulla progettazione partecipata. Contiene articoli che descrivono esperienze con la progettazione partecipata, nonché un confronto tra RAD e progettazione partecipata. Kruchten (**2003**) fornisce una buona introduzione al RUP. Robillard e Kruchten (**2003**) descrivono una versione adattata dal punto di vista accademico di RUP.

Ci sono molti libri sui metodi agili. Il libro standard su XP è del suo inventore, Beck (**2000**). Un buon volume complementare è (Jeffries *et al.*, **2001**). Altri metodi agili includono Scrum (Schwaber e Beedle, **2002**) e la famiglia di metodologie Crystal (Cockburn, **2002**). Per un confronto di una serie di metodi agili, vedere (Abrahamsson *et al.*, **2002**). Boehm e Turner (**2003**) confrontano metodi basati sulla pianificazione e metodi agili e danno consigli su quando utilizzare quale tipo di metodo. Le esperienze con metodi agili nelle grandi

organizzazioni sono descritte in (Lindvall *et al.*, **2004**). (McDowell *et al.*, **2006**) discutono i vantaggi della programmazione a coppie.

Una breve introduzione all'architettura basata su modelli (MDA) è data in (Meservy e Fenstermacher, **2005**). Il sito web di OMG (**www.omg.org/mda**) contiene molte informazioni aggiuntive su MDA.

Lehman e Belady (**1985**) offrono una panoramica dei loro primi lavori sulle leggi dell'evoluzione del software. (Lehman *et al.*, **1997**) e (Cook *et al.*, **2006**) forniscono una prospettiva aggiornata. La formulazione data in questo capitolo si basa su (Lehman *et al.*, **1997**).

Una visione di fabbrica dello sviluppo del software è stata suggerita alla prima conferenza sull'ingegneria del software (McIlroy, **1968**). Il termine "fabbrica del software" è spesso associato anche agli sforzi giapponesi per migliorare la produttività dello sviluppo del software (Cusumano, **1989**). La nozione di linee di prodotti software è emersa negli anni '80 come un modo per aumentare l'economia di scala. Clements e Northrop (**2002**) e (Pohl *et al.*, **2005**) forniscono una discussione approfondita sull'ingegneria della linea di prodotti software.

Osterweil (**1987**) ha lanciato l'idea di descrivere i processi di sviluppo del software come programmi. Valutazioni critiche di questo punto di vista sono fornite in (Lehman, **1987**), (Curtis *et al.*, **1987**) e (Curtis, **1989**). Le tendenze nella modellazione dei processi software sono descritte in (Fuggetta e Wolf, **1996**) e (Cugola e Ghezzi, **1998**).

Esercizi

1. Descrivere il modello a cascata dello sviluppo del software.

2. Descrivere l'approccio dello sviluppo rapido di applicazioni (RAD) allo sviluppo del software.

3.Discutere le principali differenze tra la prototipazione e lo sviluppo incrementale.

4.Discutere le principali differenze tra sviluppo incrementale e RUP.

5. Discutere la legge del cambiamento continuo.

6.In che modo il modello a spirale sussume la prototipazione, lo sviluppo incrementale e il modello a cascata?

7.Spiegare le pratiche di XP "programmazione di coppia" e "refactoring".

8. Cos'è l'MDA?

9. Che cos'è una linea di prodotti software?

10. Qual è lo scopo principale di avere una descrizione esplicita del processo di sviluppo del software in un modello di processo?

11. Che cos'è l'emanazione del processo?

12. Discutere i valori chiave del movimento agile.

13. ♠Supponiamo di essere coinvolti in un grande progetto riguardante lo sviluppo di un sistema di pianificazione del paziente per un ospedale. Puoi optare per una delle due strategie. La prima strategia consiste nell'iniziare con un'analisi approfondita dei requisiti degli utenti, dopodiché il sistema viene costruito in base a tali requisiti. La seconda strategia inizia con una fase di analisi dei requisiti meno completa, dopo la quale viene sviluppata una versione pilota. Questa versione pilota è installata in alcuni piccoli reparti. L'ulteriore sviluppo del sistema è guidato dall'esperienza acquisita lavorando con la versione pilota. Discuti i pro ei contro di entrambe le strategie. Quale strategia preferisci?

14. ♡Considera il progetto del sistema di pianificazione del paziente menzionato nell'Esercizio 13. In quali condizioni opteresti per un approccio agile per questo progetto?

15. Discutere i meriti relativi della prototipazione usa e getta come un modo per suscitare i "veri" requisiti dell'utente e la prototipazione come metodo di sviluppo evolutivo.

16. In che modo la nozione di linea di prodotti software può avere un impatto sulla struttura del processo di sviluppo del software?

17. ♡La manutenzione del software aumenta l'entropia del sistema. Discutere i possibili modi per contrastare questo effetto.

18. ♡Uno dei motivi per cui si utilizzano approcci basati sulla pianificazione nei progetti di sviluppo del software è che il piano fornisce una struttura per misurare l'avanzamento del progetto. Pensi che questa misura sia adeguata? Riesci a pensare a modi migliori per misurare i progressi?

19. ♡Discutere le differenze tra RAD e progettazione partecipata (vedi (Carmel *et al.*, **1993**)).

20. ♠Descrivere il processo di ingegneria dei requisiti illustrato nella **Figura 9.1** in una notazione simile a un linguaggio di programmazione. Sii il più preciso possibile. Discutere i vantaggi e i limiti della descrizione del processo risultante.

21. ♠Descrivere il processo di ingegneria dei requisiti illustrato nella **Figura 9.1** in un diagramma di transizione di stato. Discutere i vantaggi e i limiti della descrizione del processo risultante.

[1] La parola "architettura" in questa frase è in qualche modo impropria. Lo sviluppo guidato da modelli (MDD) riflette meglio di cosa si tratta.

[2] In una revisione, un documento (come un pezzo di codice o un progetto) viene prima studiato individualmente da un paio di revisori. I problemi riscontrati vengono quindi discussi dai revisori e dall'autore del documento (cfr. Sezione 13.4.2).